

*Software per la gestione di una biblioteca
universitaria*

Identificativo progetto: G8

Gruppo numero: 8

Nome documento: Progetto Ingegneria del software

Data documento: 8/12/2025

Versione: 2.0

Team: Alessandro Picariello, Lorenzo Trovato, Matteo Iandiorio, Daniele Manzo

1. Introduzione

1.1 Obiettivi

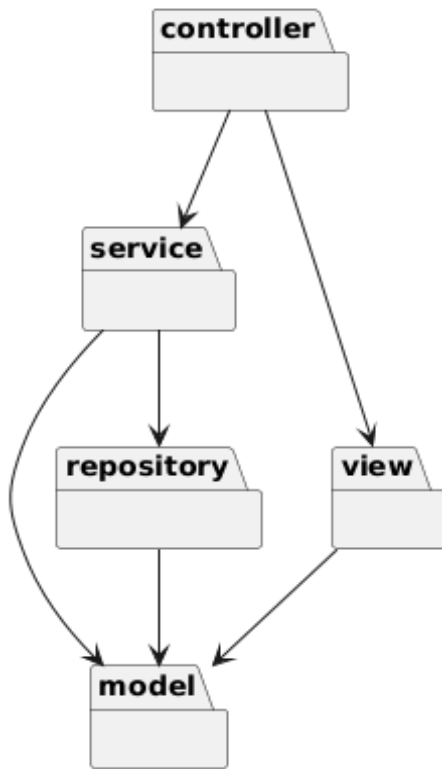
Il presente documento si prefigge l'obiettivo di esplicitare le scelte di design effettuate, nonché di fornire la descrizione dei modelli statici e dei modelli dinamici e dell'architettura del sistema nella sua totalità.

1.2 Architettura del sistema

Il sistema si divide in 5 moduli chiave:

- 1) **Model**: il *model* fornisce i metodi per accedere ai dati utili all'applicazione
 - Componenti: InterfaceID, Dati, Libro, Utente e Prestito.
- 2) **View**: la *view* visualizza i dati contenuti nel model e si occupa dell'interazione con utenti e agenti
 - Componenti: InterfaceView<T>, BaseView<T>, DatiBaseView<T>, LibroView, UtenteView, PrestitoView.
- 3) **Controller**: il *controller* riceve i comandi dell'utente (in genere attraverso la view) e li attua modificando lo stato degli altri due componenti
 - Componenti: InterfaceController, BaseController<T>, CrudController<T>, LibroController, UtenteController, PrestitoController.
- 4) **Repository**: il repository gestisce l'accesso e la memorizzazione dei dati, isolando al resto del sistema i dettagli dell'IO (Input/Output)
 - Componenti: InterfaceGestoreIO<T> e GestoreFile<T>, InterfaceRepository<T>, Repository<T>
- 5) **Service**: il service contiene le regole e la logica di business del sistema
 - Componenti: InterfaceService<T>, BaseService<T>, LibroService, UtenteService, PrestitoService.

Diagramma dei package



- **service -> repository** : Il service si occupa di salvare, modificare, cercare ed eliminare gli oggetti.

- **controller -> service** : Il controller si occupa di gestire e coordinare le richieste dell'utente, tuttavia non contiene la logica di business.

- **controller -> view** : il controller gestisce la view, come avviene nel modello MVC.

- **view -> model** : la view ha bisogno di accedere ai dati, di cui il model è a conoscenza. L'interazione diretta serve anche ad evitare ulteriori dipendenze con il repository o con il service.

- **repository -> model** : il repository deve salvare e/o caricare i dati del model.

- **service -> model** : il service deve applicare le logiche di business sul model.

Pattern principali usati:

Model-View-Controller MVC
Repository

Principi di buona progettazione impiegati:

- ☐ DRY
- ☐ KISS
- ☐ SINE
- ☐ YAGNI
- ☐ SoC
- ☐ Principio della minima sorpresa
- ☐ Regola del boy-scout

Principi di buona progettazione orientata agli oggetti impiegati:

- ☐ Principio della singola responsabilità
- ☐ Principio di segregazione delle interfacce
- ☐ Principio di sostituzione di Liskov
- ☐ Principio aperto/chiuso
- ☐ Principio di inversione della dipendenza

➤ **Viste architetturelle impiegate:**

Vista logica: definizione dei moduli costituenti e delle relazioni che intercorrono tra essi, mediante l'uso del diagramma delle classi

Vista di processo: definizione delle interazioni tra i componenti del sistema, a tempo di esecuzione.

➤ **Stili architetturelle adottati:**

Model-View-Controller Design Pattern: stile architetturelle che prevede la separazione del software in tre componenti: Model (adibito alla gestione dei dati), View (preposto alla visualizzazione all'utente) e Controller (designato al coordinamento).

Repository Design Pattern: stile architetturelle che consente di astrarre l'accesso e la gestione dei dati, separandoli dalla logica applicativa del software.

➤ **Approcci alla scomposizione impiegati:**

Approccio funzionale o procedurale: Suddivisione delle funzionalità da realizzare in operazioni più semplici.

Approccio object-oriented: Suddivisione in classi, tesa alla descrizione del dominio del problema.

➤ **Principi generali di buona progettazione impiegati:**

DRY (Don't Repeat Yourself) e KISS (Keep It Simple, Stupid!): L'uso del polimorfismo e dei generics consente di ridurre la ridondanza nel codice, favorendo la flessibilità e la compattezza.

Esempi all'interno del codice:

È stata evitata la scrittura dei metodi **salva**, **cerca**, **elimina** per ognuna delle classi del model (Libri, Utenti, Prestiti), favorendo una singola implementazione all'interno delle classi generiche **Repository<T>** e **BaseService<T>**.

SINE (Simple Is Not Easy): È stato effettuato un processo di adattamento del codice, finalizzato al conseguimento di una maggiore semplicità di utilizzo. Tuttavia, il trade-off è rappresentato dalla crescita della complessità del codice, dovuto precipuamente all'impiego dei generics.

Principio della minima sorpresa: È stata adoperata la convenzione di denominazione standard di Java, a cui si è affiancata un'accurata scelta nell'attribuzione di identificatori tesi a riflettere le funzionalità del codice. Inoltre, è stata ridotta al minimo la ridondanza.

YAGNI (You Aren't Going to Need It): È stata effettuata l'eliminazione di diversi metodi, al fine di evitare l'implementazione di funzionalità superflue.

Regola del boy-scout: Nelle diverse fasi di manutenzione del codice è stato effettuato un graduale affinamento della qualità del codice, con particolare riguardo per il miglioramento della chiarezza del codice.

➤ **Principi di buona progettazione orientata agli oggetti:**

Principio della singola responsabilità: È stata operata una suddivisione delle classi, in maniera tale che ciascuna di esse effettui un compito ben preciso.

Principio di segregazione delle interfacce: L'inserimento di un'interfaccia in ogni macroarea è finalizzato a ridurre la presenza di lunghe catene di dipendenze.

- Il **BaseService** non dipende dalla classe concreta **Repository**, ma dall'interfaccia **InterfaceRepository<T>**.
- Il **Repository** non dipende dalla classe concreta **GestoreFile**, ma dall'interfaccia **InterfaceGestoreIO<T>**.

Principio di sostituzione di Liskov: Le classi astratte sono state implementate in modo tale che gli oggetti possano esser sostituiti con istanze dei loro sottotipi, senza determinare alterazioni alla correttezza del programma

Principio aperto-chiuso: L'overriding dei metodi della superclasse all'interno delle sottoclassi e l'incapsulamento consentono di apportare modifiche alle classi di base, evitando che si riflettano sulle classi derivate.

➤ **Separation of Concerns (SoC - Separazione degli Interessi)**

È stata effettuata una separazione dell'applicazione in cinque livelli: **View** , **Controller** , **Service**, **Repository**, **Model**.

2. Diagramma delle classi

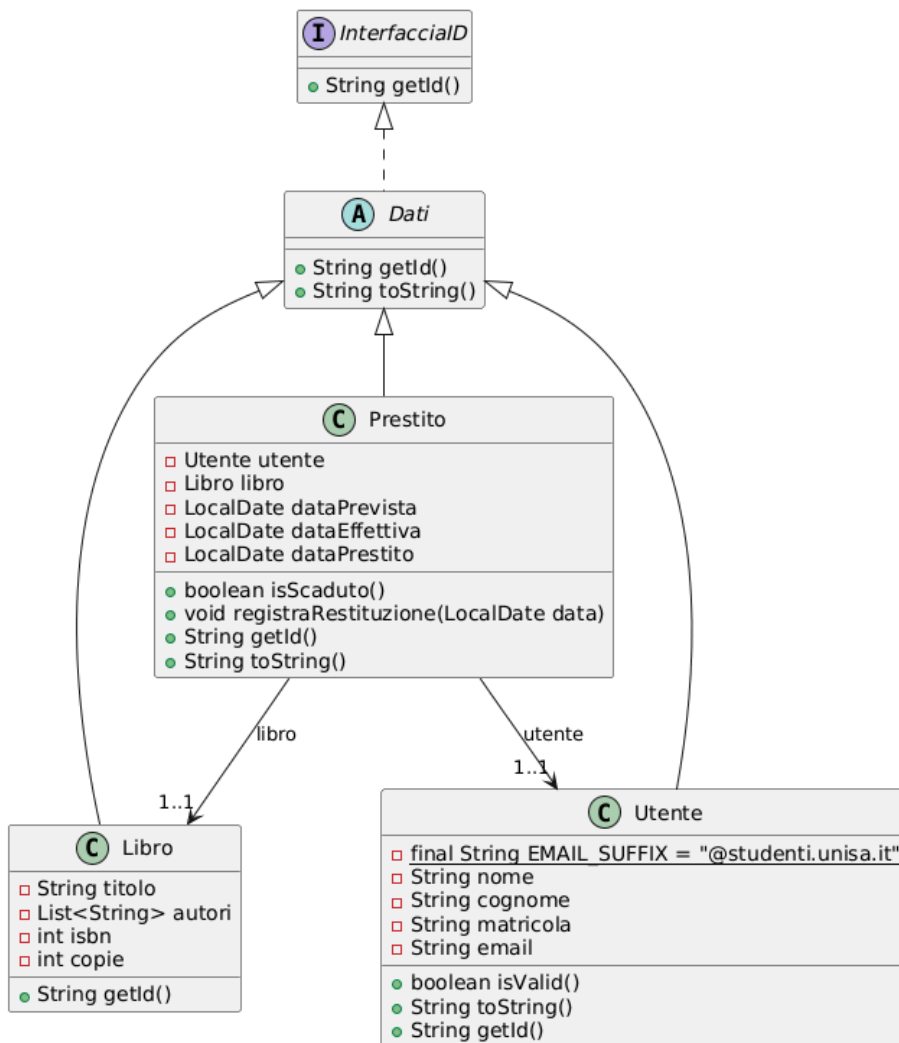
Nota1: Il diagramma delle classi complessivo non è stato aggiunto per agevolare la comprensione.

Nota2: Nei diagrammi delle classi sono state omesse alcune dipendenze, i getter/setter e i costruttori per favorire la lettura.

Model

Il modulo Model si occupa di rappresentare le entità con le quali opera il sistema.

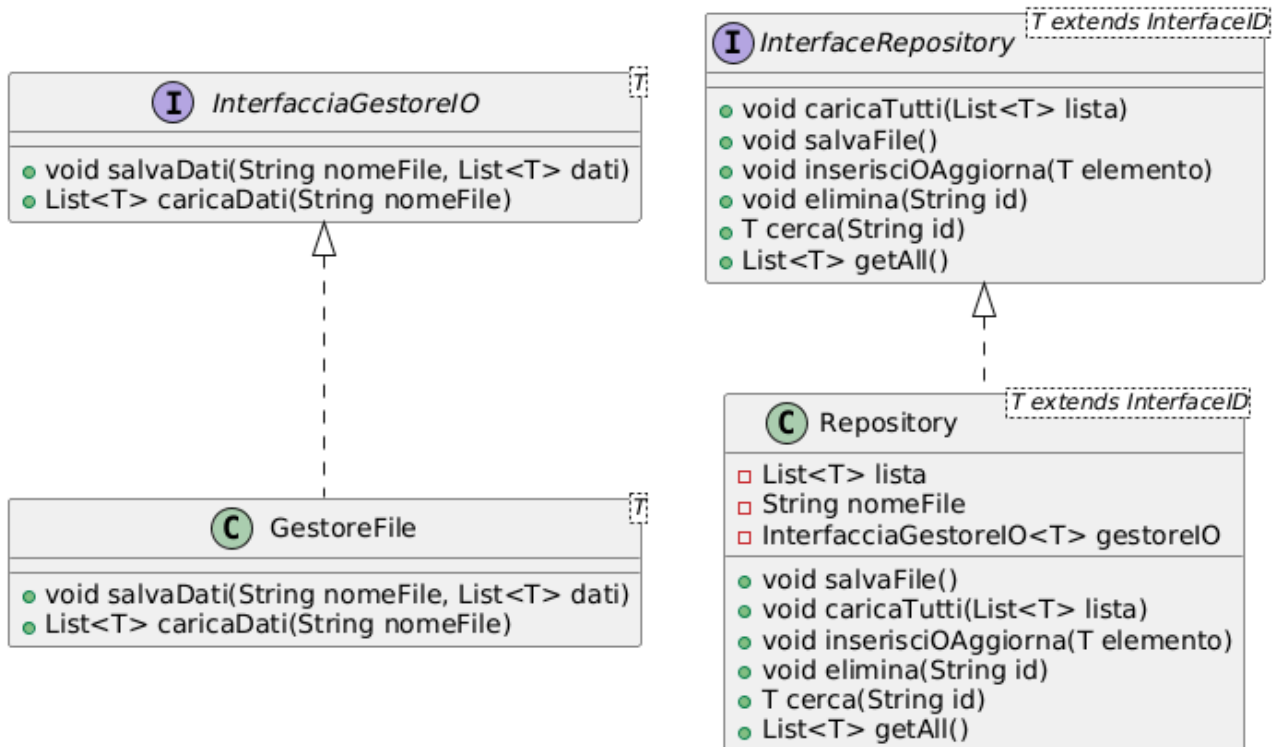
Livello Modello Dati



Repository

Il modulo Repository è deputato all'accesso e al salvataggio dei dati, garantendone la persistenza.

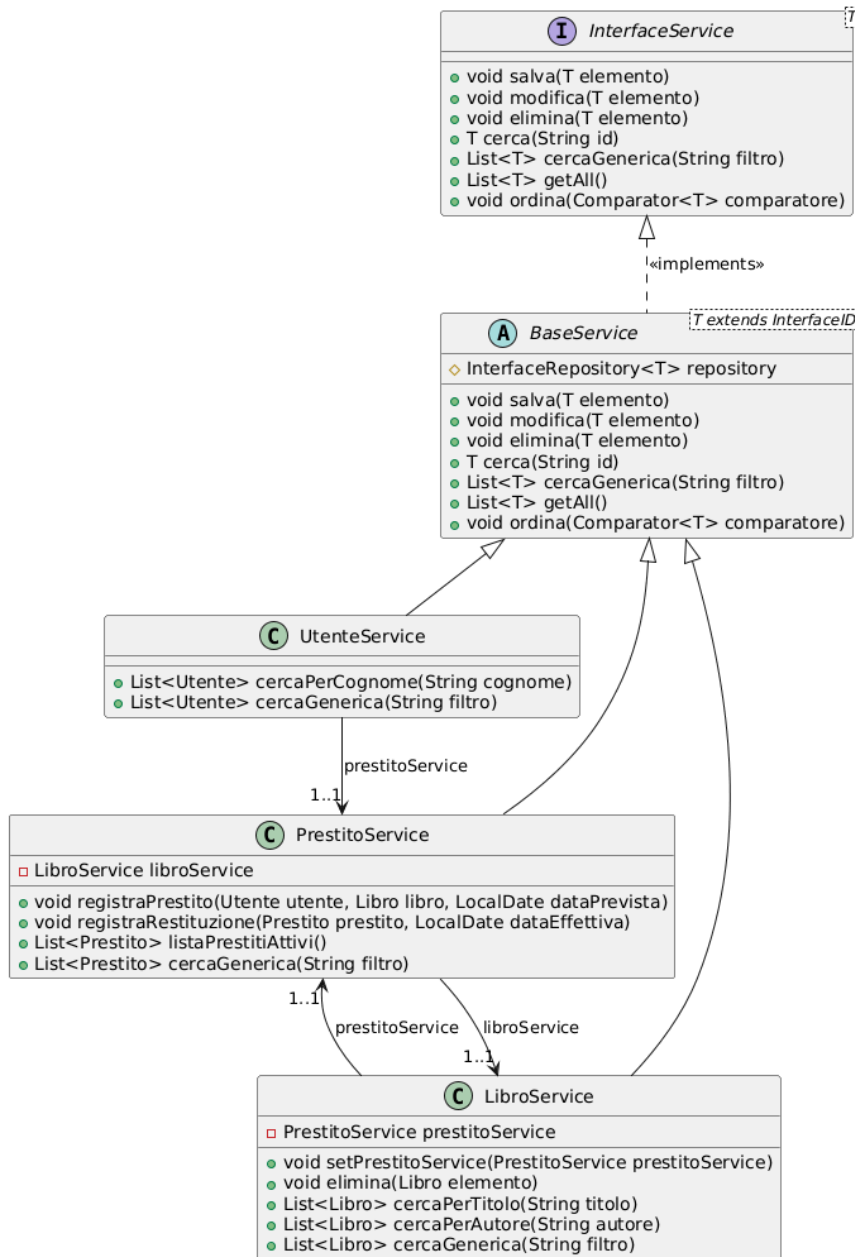
Livello Repository



Service

Il modulo Service si occupa dell'implementazione delle effettive funzionalità messe a disposizione dal sistema, nonché dell'orchestrazione dell'applicazione stessa. Opera sui dati forniti dal Repository.

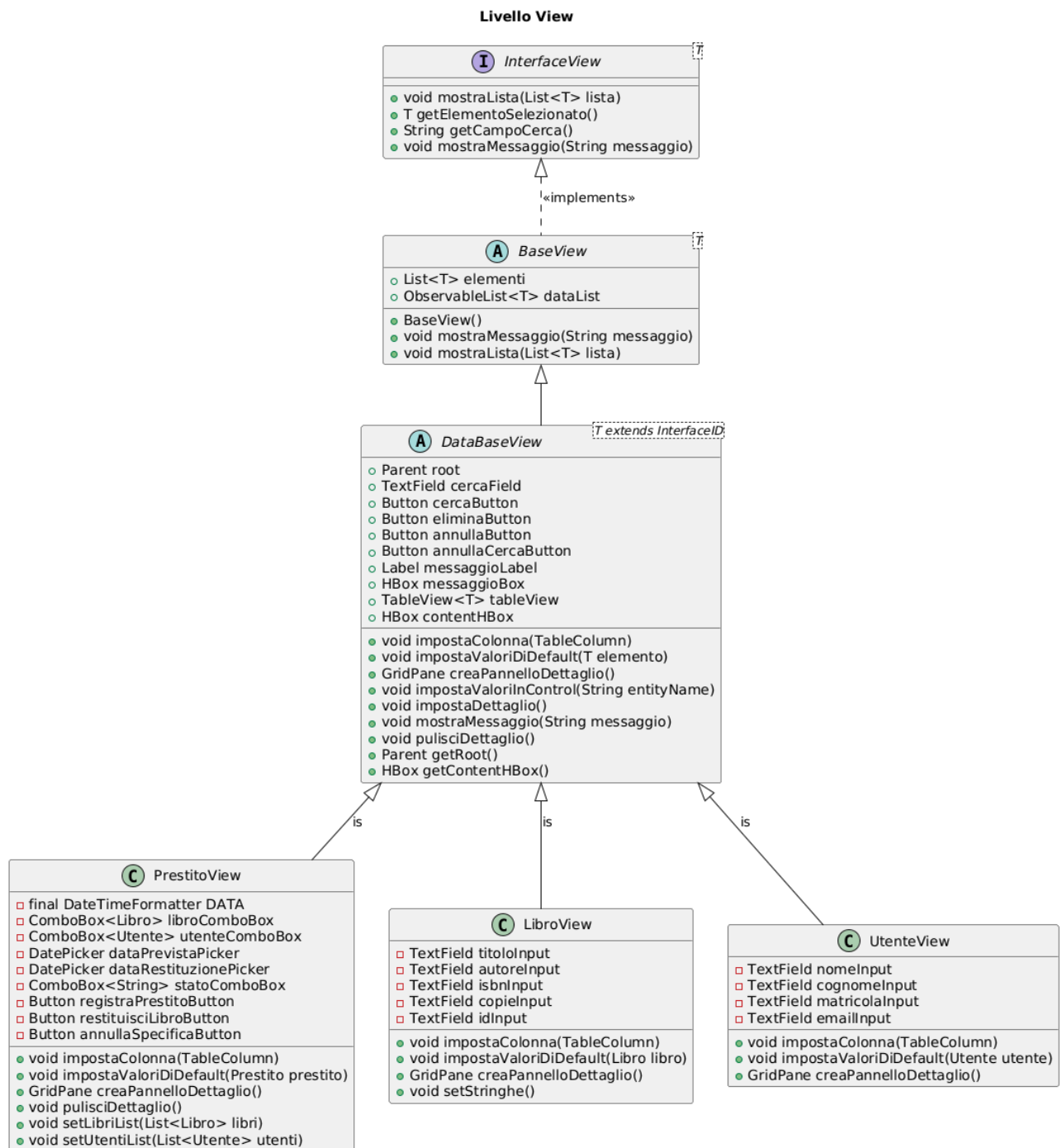
Livello Service



Perchè PrestitoService dipende da LibroService ma non UtenteService? Perchè quando un Prestito viene registrato il sistema deve diminuire le copie disponibili e, viceversa, quando viene restituito deve aumentare le copie disponibili. Diversamente, le informazioni dell'utente rimangono invariate.

View

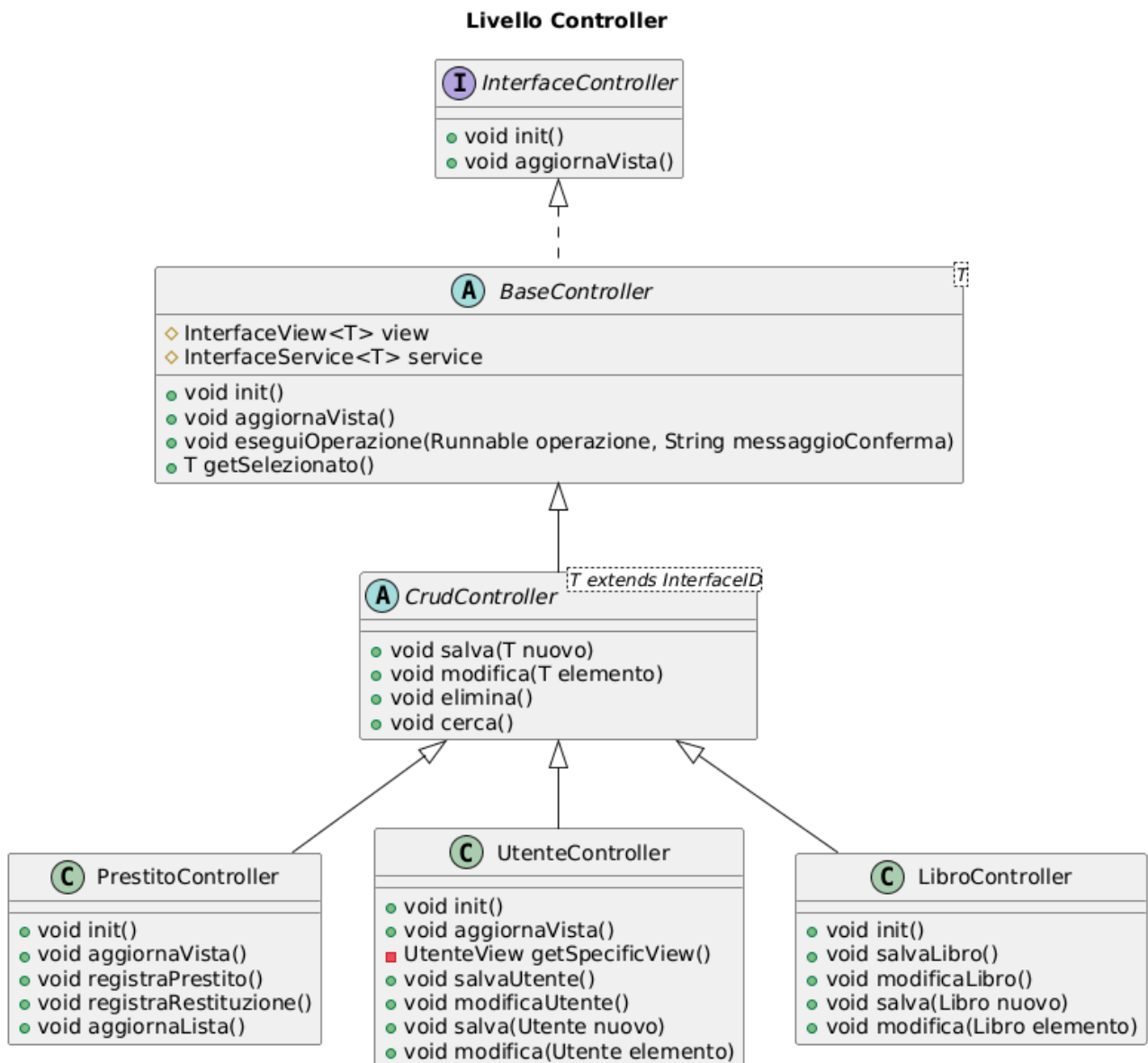
Il modulo View è responsabile dell'interfaccia utente, della visualizzazione dei dati e della raccolta dell'input inserito dal gestore del sistema.



Controller

Il modulo Controller si occupa di gestire e coordinare il flusso dell'applicazione, fungendo da intermediario tra la View ed il Service. Deve rispondere agli eventi

generati in risposta alle azioni intraprese dal gestore del sistema, invocare i metodi del Service e aggiornare la View.



3. Modello dinamico

Il Modello Dinamico descrive il comportamento del sistema e come gli oggetti interagiscono tra loro nel tempo per completare i compiti.

3.1 Diagramma di sequenza

Prestito

Registrazione prestito

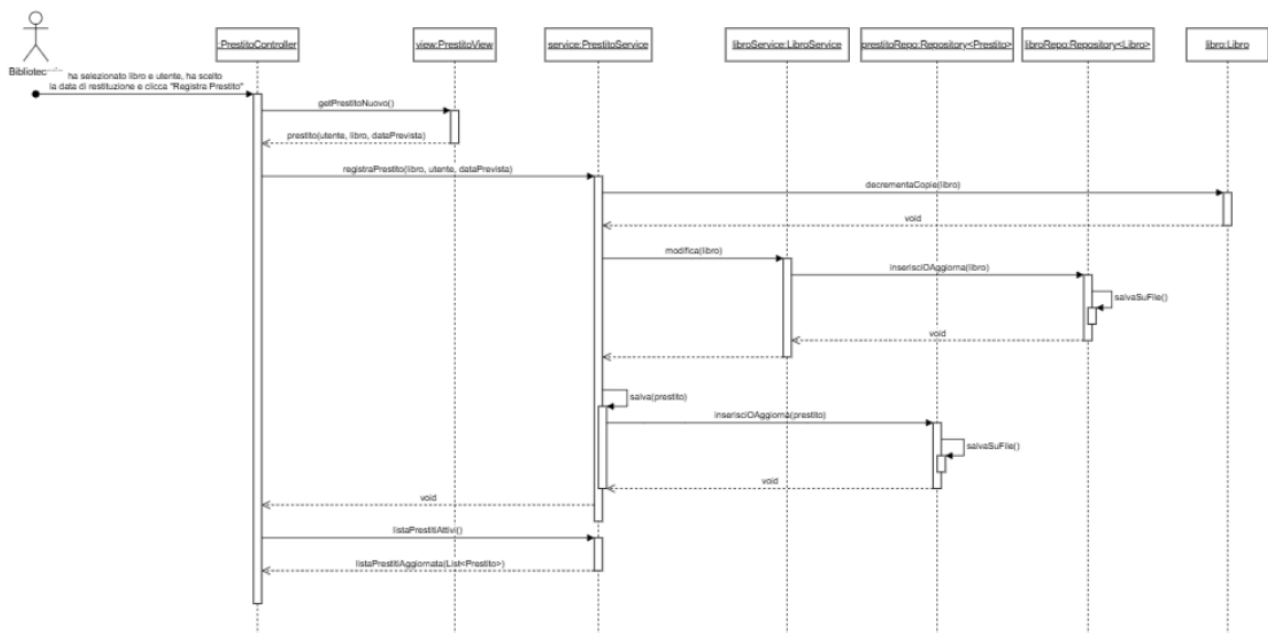


Diagramma raffigurante la fase di registrazione del prestito:

1. Il **:PrestitoController** ottiene i dati del prestito da registrare dalla View.
2. Il Controller effettua la chiamata del metodo **registraPrestito(libro, utente, dataPrevista)** su **:PrestitoService**.
3. Il **:PrestitoService** decrementa il numero di copie disponibili del libro che si intende concedere in prestito attraverso la chiamata

decrementaCopie(libro), e delega la modifica al **:LibroService** attraverso il metodo **modifica(libro)**.

4. Il **:LibroService** invoca il **:Repository<Libro>** per effettuare il salvataggio fisico dei dati su file.
5. Il **:PrestitoService** procede a salvare il prestito invocando il **:Repository<Prestito>** mediante la chiamata al metodo **inserisciOAggiorna(prestito)**.
6. Viene salvato il nuovo oggetto **prestito**.
7. Il flusso di controllo torna al **:PrestitoController**, che richiede al **:PrestitoService** la lista aggiornata dei prestiti attivi, invocando **listaPrestitiAttivi()**.

Restituzione prestito

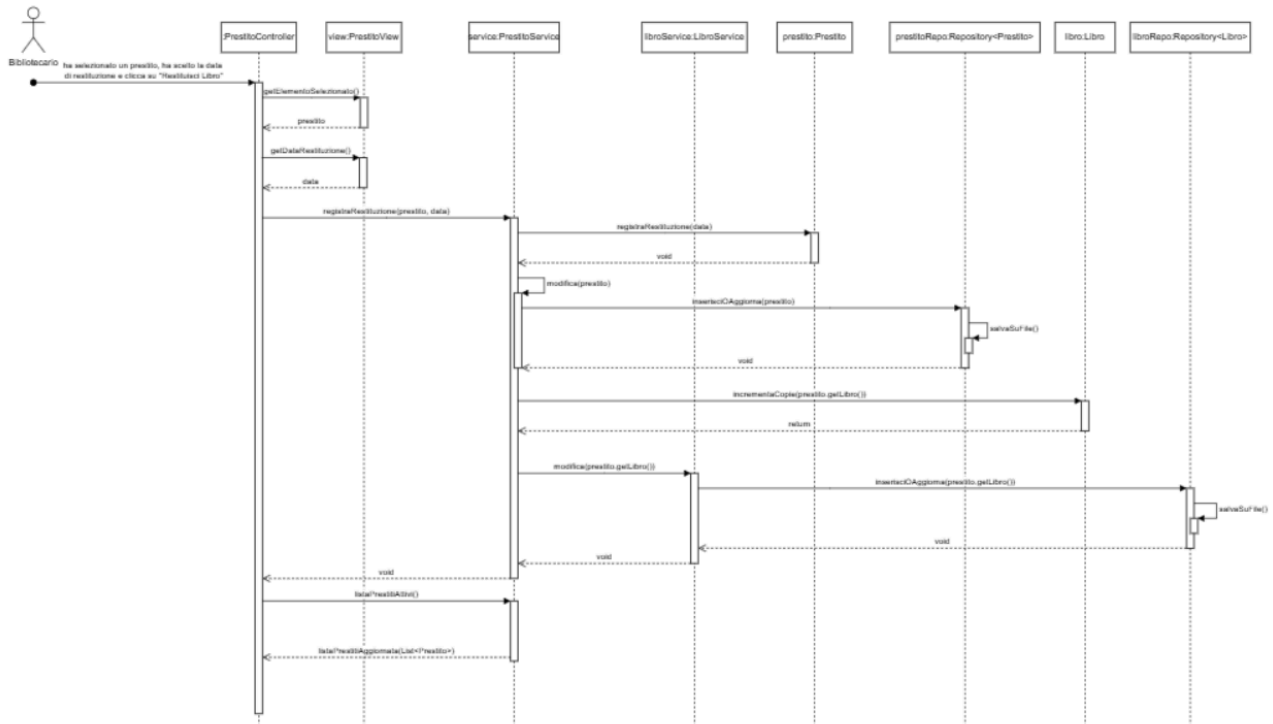


Diagramma raffigurante la fase di registrazione della restituzione:

1. Il **:PrestitoController** raccoglie i dati del prestito e della data di restituzione dalla View mediante i metodi **getElementoSelezionato()** che restituisce il prestito selezionato e **getDataRestituzione()** che restituisce la data inserita.
2. Il **:PrestitoService** registra la restituzione mediante il metodo **registraRestituzione(prestito, data)** ed esegue una transazione a tre fasi:
 - Invoca **registraRestituzione(data)** sull'oggetto prestito.
 - Invoca il metodo **modifica(prestito)** su stesso delegando poi il salvataggio effettivo al **:Repository<Prestito>**.
 - Infine invoca **prestito.getLibro().incrementaCopie()** e delega l'aggiornamento del libro al **:LibroService**.
3. Il **:PrestitoController** richiede l'aggiornamento della lista dei prestiti, invocando il metodo **aggiornaPrestiti()**. Viene restituita la lista dei prestiti attivi, provvista delle modifiche apportate.

Libro

Cerca Libro id

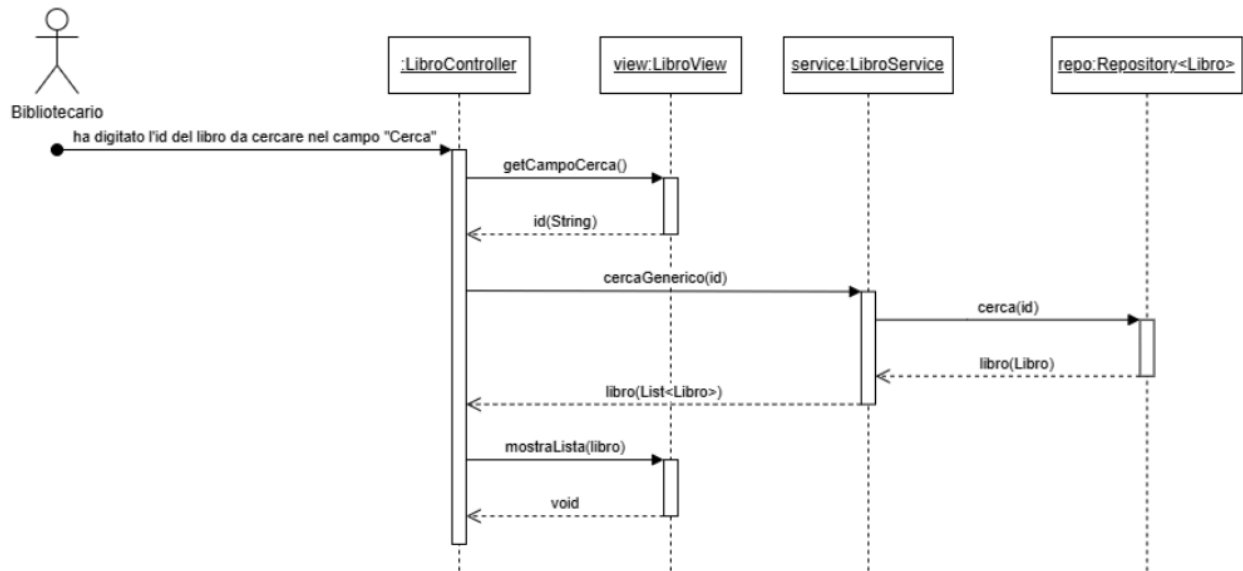


Diagramma raffigurante la fase di ricerca di un libro tramite id:

1. Il **:LibroController** ottiene l'id del libro dalla View.
2. Chiama **cerca(id)** sul **:LibroService**.
3. Il **:LibroService** chiama **cerca(id)** sul **:Repository<Libro>**.
4. Il Repository restituisce il libro corrispondente.
5. Il **:PrestitoController** ottiene la lista dei libri dalla View.

Cerca libro filtrato

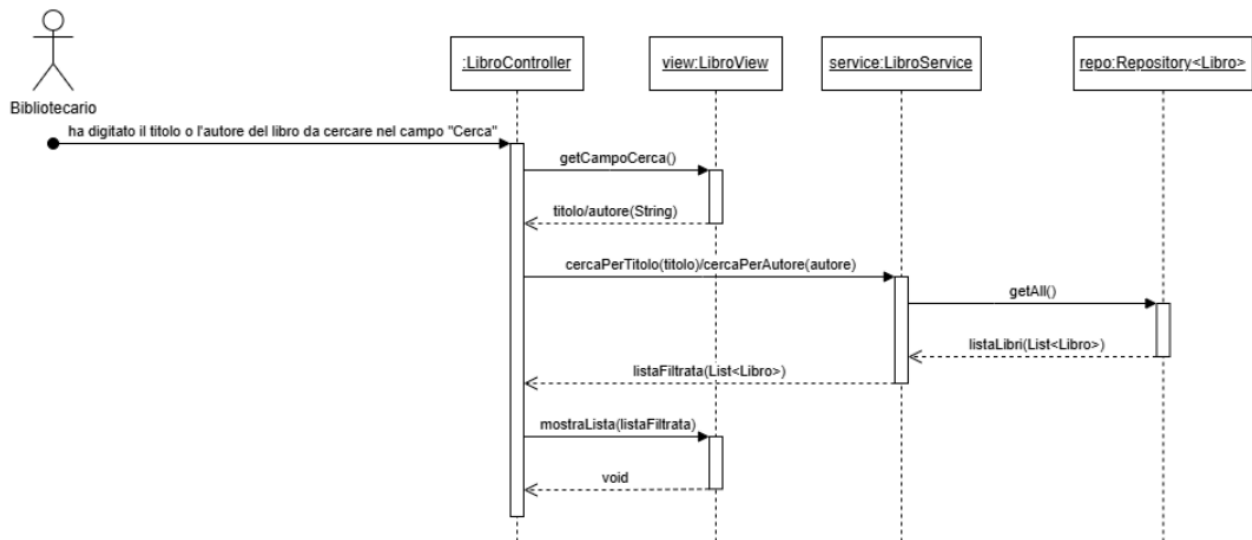


Diagramma raffigurante la fase di ricerca filtrata di un libro:

1. Il **:LibroController** ottiene la stringa inserita contenente o il titolo o l'autore del libro dalla View mediante il metodo **getCampoCerca()**.
2. Il Controller chiama il metodo **cercaPerTitolo(titolo)/cercaPerAutore(autore)** sul **:LibroService**.
3. Il **:LibroService** chiama **getAll()** sul **:Repository<Libro>**.
4. Il Repository restituisce la lista dei libri corrispondenti, successivamente Service cerca la stringa tra i generics e restituisce la lista filtrata..
5. Il Controller chiama **mostraLista(listaFiltrata)**, ottenendo la lista dei libri filtrata dalla View.

Elimina libro

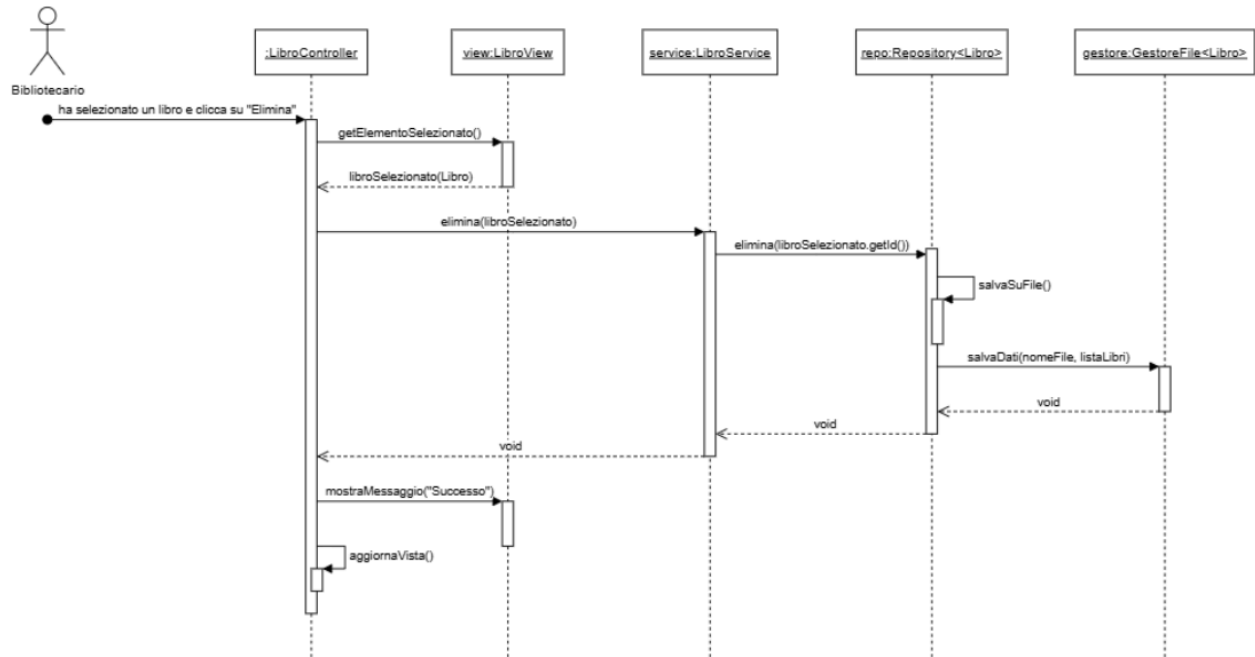


Diagramma raffigurante la fase di eliminazione di un libro:

1. Il **:LibroController** ottiene il libro da eliminare dalla View mediante il metodo **getElementoSelezionato()**.
2. Il **:LibroController** chiama il metodo **elimina(libro)** sul **:LibroService**.
3. Il Service chiama **elimina(libro.id)** sul **:Repository<Libro>**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la rimozione.
5. Il Controller notifica l'esito e aggiorna la vista.

Inserimento libro

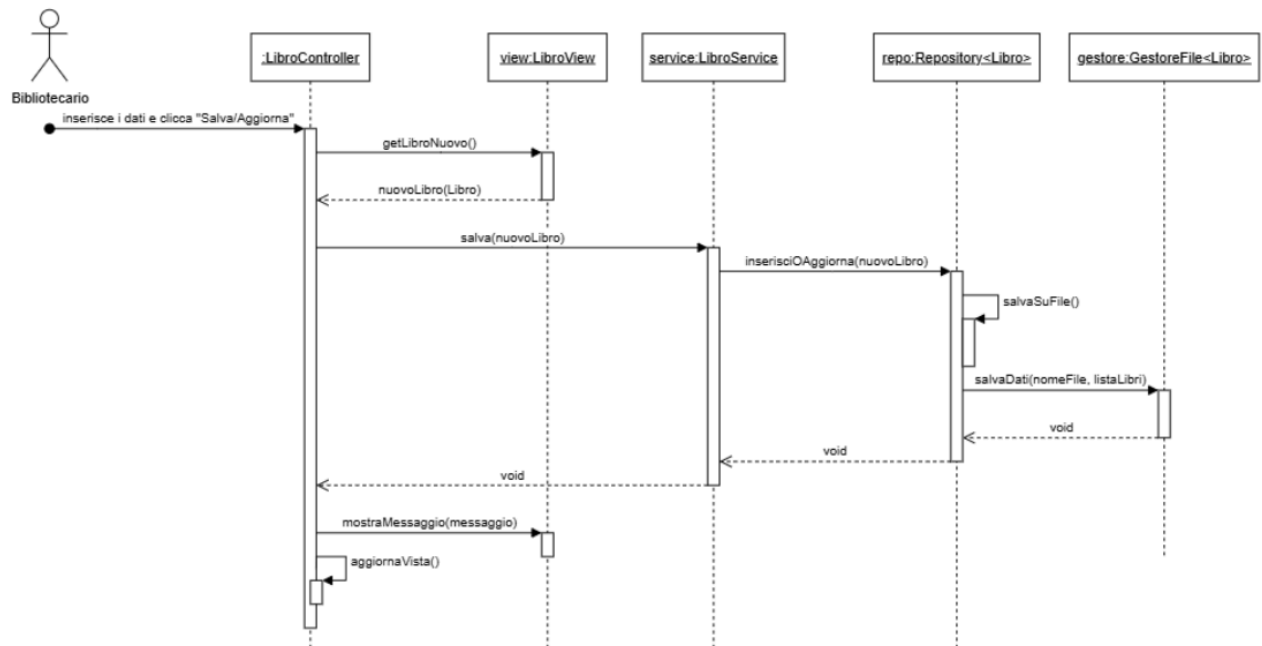


Diagramma raffigurante la fase di aggiunta di un nuovo libro:

1. Il **:LibroController** ottiene il libro da aggiungere dalla View mediante il metodo **getLibroNuovo()**.
2. Chiama **salvaLibro(nuovoLibro)** sul **:LibroService**.
3. Il Service chiama **inserisciOAggiorna(nuovoLibro)** sul **:Repository<Libro>**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente l'aggiunta.
5. Il Repository chiama **salvaDati(nomeFile, listaLibri)** sul **:GestoreFile<Libro>**.
6. Il Controller notifica l'esito e aggiorna la vista.

Modifica libro

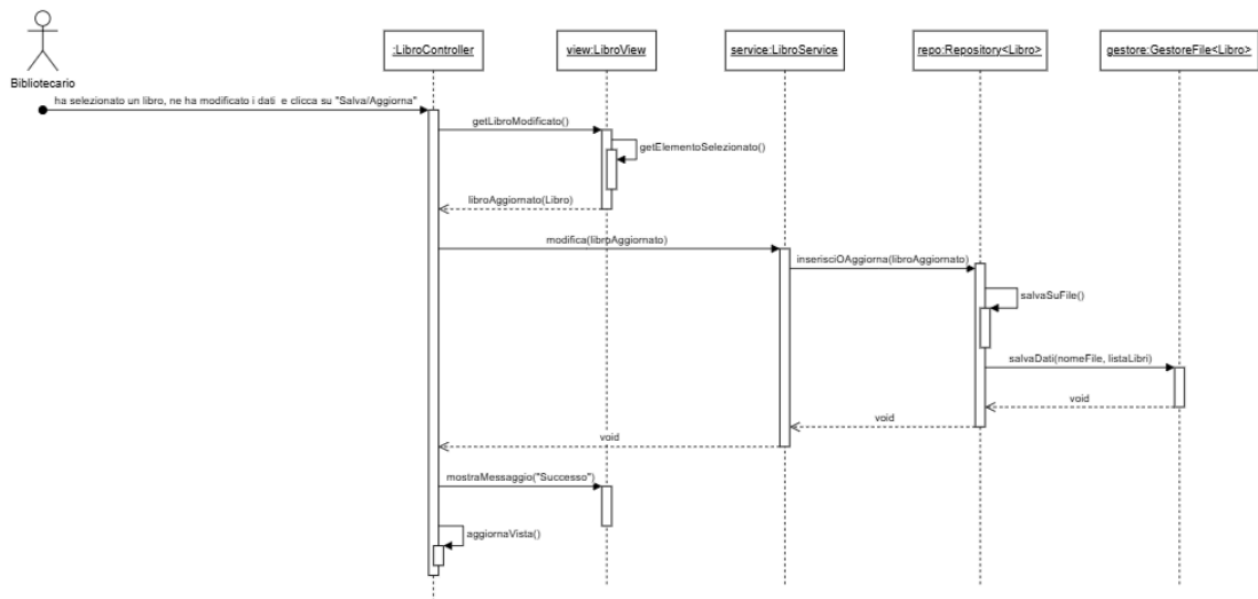


Diagramma raffigurante la fase di modifica di un libro:

1. Il **:LibroController** ottiene il libro da modificare dalla View mediante il metodo **getLibroModificato()**.
2. Chiama **modifica(libro)** sul **:LibroService**.
3. Il Service chiama **InserisciOAggiorna(libro)** su **:Repository<Libro>**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la modifica.
5. Il Repository chiama **salvaDati(nomeFile, listaLibri)** sul **:GestoreFile<Libro>**.
6. Il Controller notifica l'esito e aggiorna la vista.

Utente

Cerca Utente Id

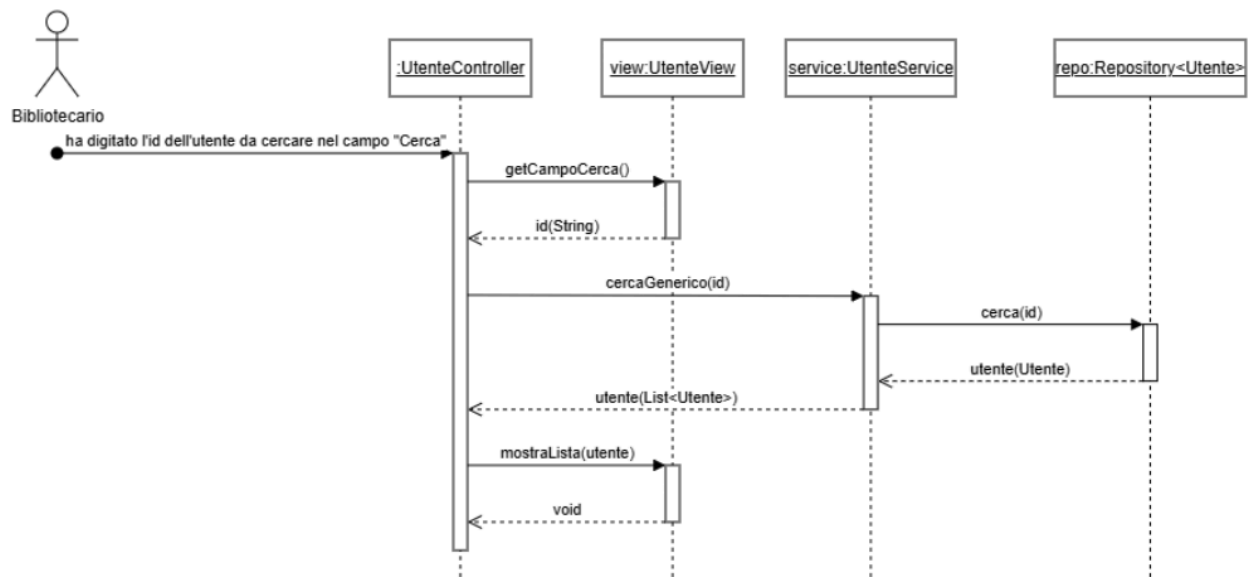


Diagramma raffigurante la fase di ricerca di un utente tramite id:

1. Il **:UtenteController** ottiene l'id del libro dalla View mediante il metodo **getCampoCerca()**.
2. Chiama **cerca(id)** sul **:UtenteService**.
3. Il Service chiama **cerca(id)** sul **:Repository<Utente>**.
4. Il Repository restituisce l'utente corrispondente.
5. La View mostra la lista degli utente, contenente quello desiderato.

Cerca utente filtrato

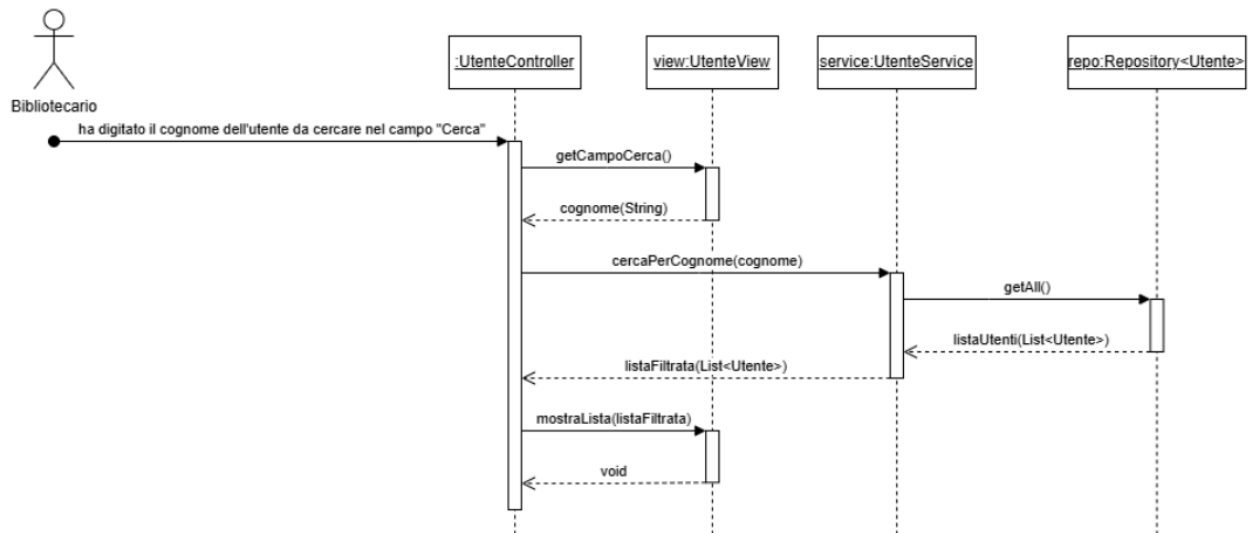


Diagramma raffigurante la fase di ricerca filtrata di un utente:

1. Il **:UtenteController** ottiene la stringa inserita contenente il cognome dell'utente dalla View mediante il metodo **getCampoCerca()**.
2. Chiama **cercaPerCognome(cognome)** sul **:UtenteService**.
3. Il Service chiama **getAll()** sul **:Repository<Utente>**.
4. Il Repository restituisce la lista degli utenti corrispondenti, successivamente il Service cerca la stringa tra i generics e restituisce la lista filtrata.
5. Il Controller chiama **mostraLista(listaFiltrata)** e View mostra la lista filtrata degli utenti.

Elimina utente

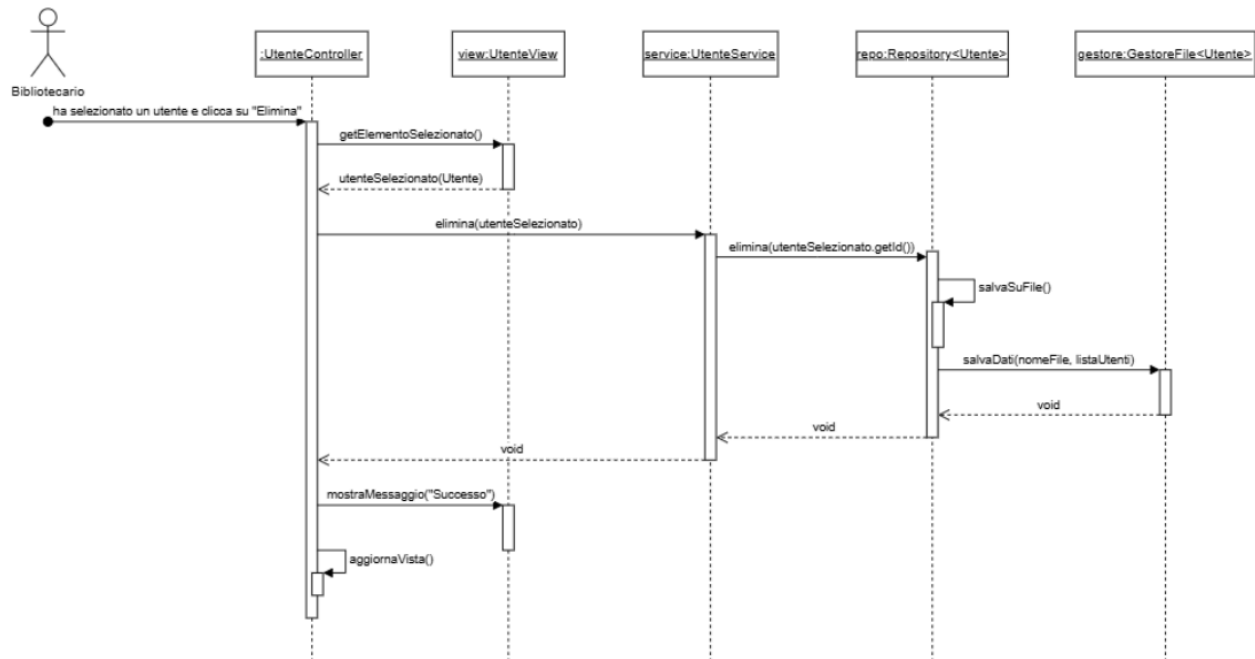


Diagramma raffigurante la fase di eliminazione di un utente:

1. Il **:UtenteController** ottiene l'utente da eliminare dalla View mediante il metodo **getElementoSelezionato()**.
2. Chiama **elimina(utente)** sul **:UtenteService**.
3. Il Service chiama **elimina(utente.id)** sul **:Repository<Utente>**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la rimozione.
5. Il Repository chiama **salvaDati(nomeFile, listaUtenti)** sul **:GestoreFile<Libro>**.
6. Il Controller notifica l'esito e aggiorna la vista.

Inserimento utente

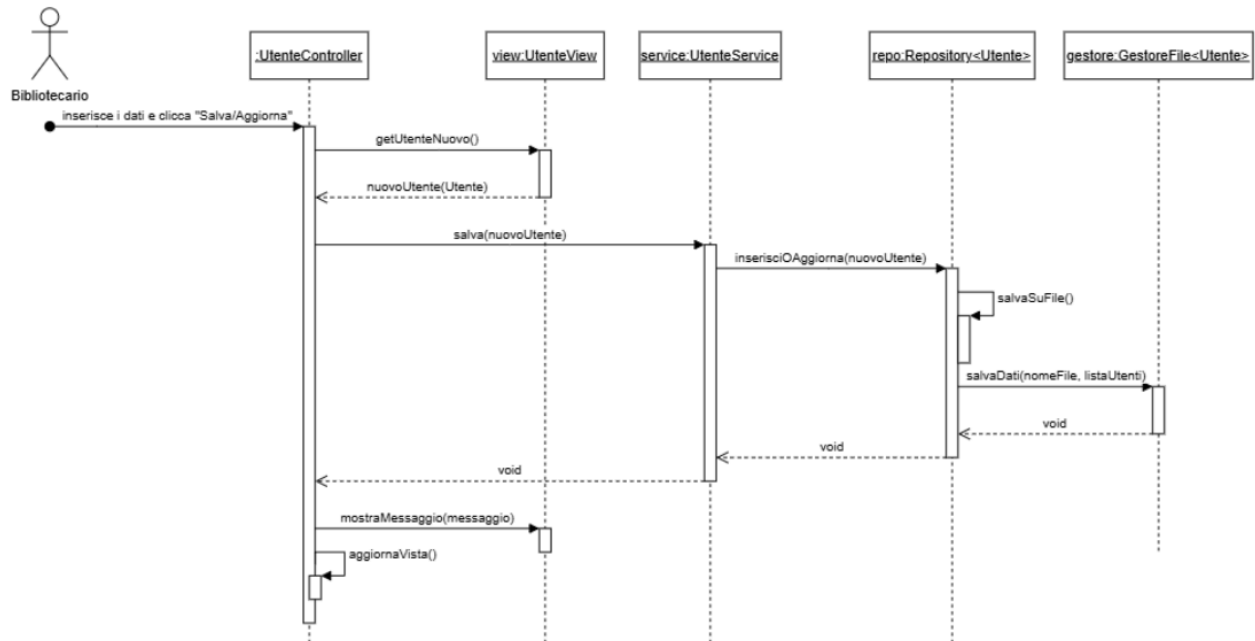


Diagramma raffigurante la fase di inserimento di un nuovo utente:

1. Il **:UtenteController** ottiene l'utente da aggiungere dalla View mediante il metodo **getUtenteNuovo()**.
2. Chiama **salvaUtente(nuovoUtente)** sul **:UtenteService**.
3. Il Service chiama **inserisciOAggiorna(nuovoUtente)** sul **:Repository<Utente>**.
4. Il Repository chiama **salvaDati(nomeFile, listaUtenti)** sul **gestore:GestoreFile**.
5. Il Repository esegue **salvaSuFile()** per rendere permanente l'aggiunta.
6. Il Controller notifica l'esito e aggiorna la vista.

Modifica utente

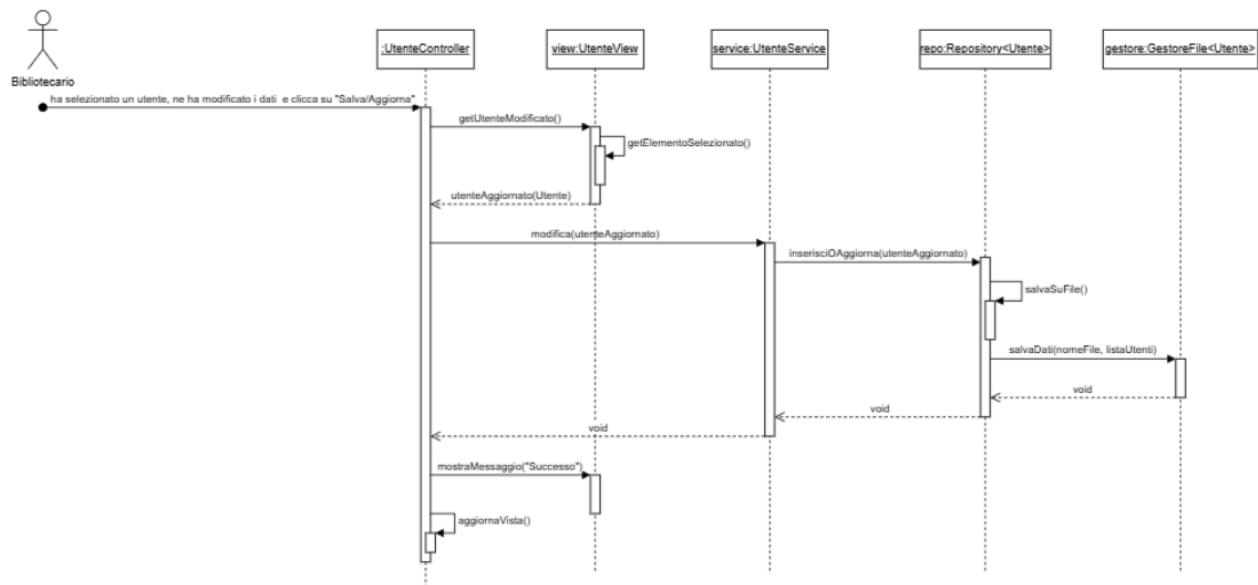


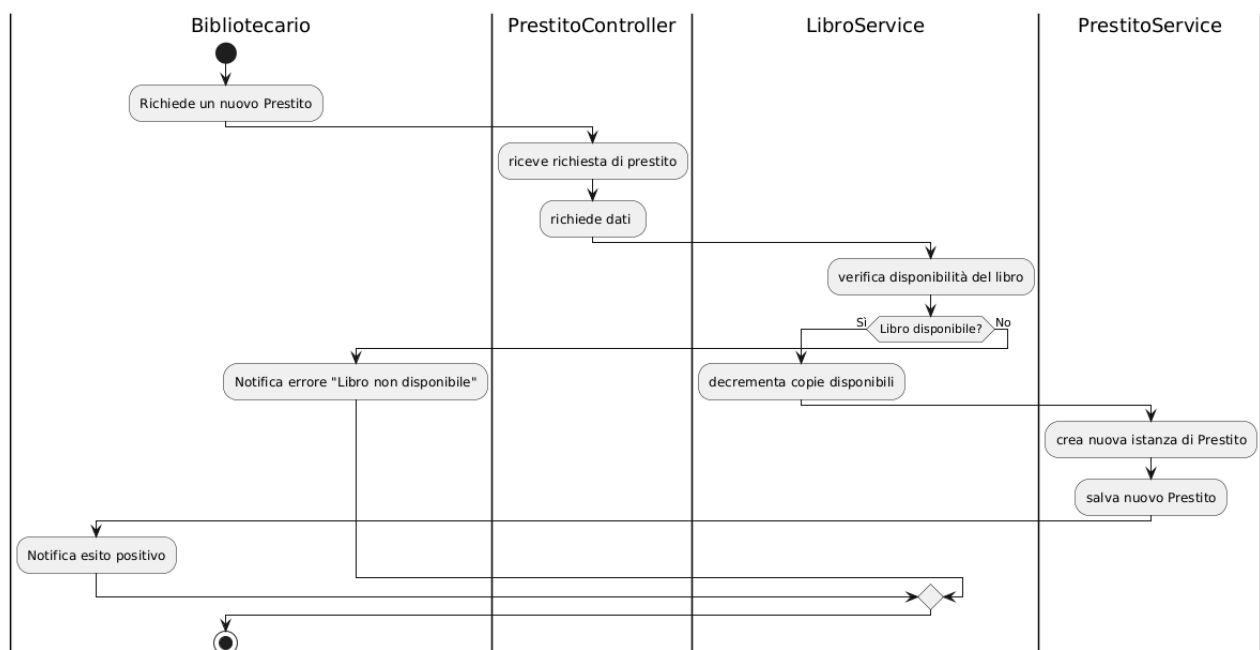
Diagramma raffigurante la fase di modifica di un utente:

1. Il **:UtenteController** ottiene l'utente da modificare dalla View mediante il metodo **getUtenteModificato()**.
2. Chiama **modifica(utente)** sul **:UtenteService**.
3. Il Service chiama **inserisciOAggiorna(utente)** sul **:Repository<Utente>**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la modifica.
5. Il Repository chiama **salvaDati(nomeFile, listaUtenti)** sul **gestore:GestoreFile**.
6. Il Controller notifica l'esito e aggiorna la vista.

3.2 Diagramma delle attività:

Il diagramma delle attività descrive la sequenza passo-passo delle azioni e le decisioni che avvengono durante l'esecuzione di un processo, di un'operazione o di un caso d'uso.

Registra Prestito



1. Inizio e Verifica

- **Richiede un nuovo prestito:** Il Bibliotecario richiede la registrazione di un nuovo prestito.
- **Riceve richiesta di prestito:** PrestitoController riceve la richiesta di registrazione del prestito.
- **Richiede dati:** Il Controller necessita dei dati necessari alla registrazione del prestito, quali l'utente richiedente (Utente), il libro di cui si deve effettuare la concessione (Libro), e la data di restituzione (Data Prevista); per tale motivo, PrestitoController deve ottenere i dati mediante un'apposita richiesta.

- **Verifica disponibilità del libro:** Il Controller deve verificare se il libro indicato sia disponibile per il prestito; a tale scopo, delega a LibroService il compito di controllare se siano presenti copie disponibili.

2. Decisione Condizionale

Il flusso si biforca basandosi sul risultato della verifica di disponibilità:

- **Ramo "No" (Libro non disponibile):**
 - Se il libro richiesto non è disponibile per il prestito (No), il controllo torna al Bibliotecario.
 - **Notifica errore "Libro non disponibile":** Viene visualizzato un messaggio volto a notificare un errore.
 - Unificazione del flusso e terminazione.
- **Ramo "Sì" (Libro disponibile):**
 - Se il Libro è disponibile per il prestito (Sì), LibroService procede.
 - **Decrementa copie disponibili:** LibroService provvede a diminuire il numero di copie disponibili (funzionalità messa a disposizione dalla chiamata del metodo decrementaCopie() sul modello del Libro).
 - Il controllo passa a PrestitoService.

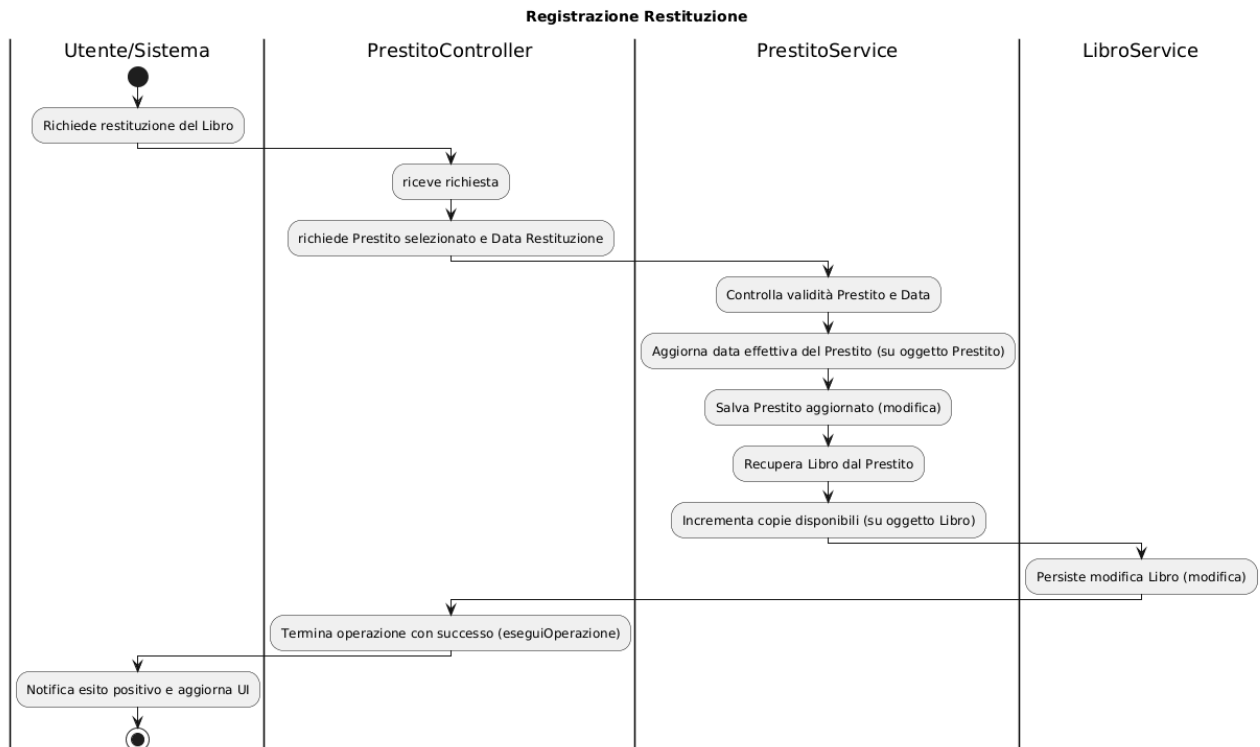
3. Finalizzazione della Transazione

- **Crea nuova istanza di Prestito:** PrestitoService crea la nuova istanza Prestito con i dati ottenuti.
- **Salva nuovo Prestito:** Il Service effettua il salvataggio dell'oggetto all'interno del Repository (rendendolo, in tal modo, persistente).
- **Notifica esito positivo:** Il flusso torna al Bibliotecario; viene notificato l'esito positivo dell'operazione.

4. Conclusione

- Unificazione e terminazione del flusso.

Registra Restituzione



1. Ricezione e Identificazione

- **Richiesta iniziale:** Il Bibliotecario avvia il processo, richiedendo la restituzione di un libro.
- **Ricezione e Raccolta Dati:** Il PrestitoController riceve la richiesta, identifica il prestito selezionato e richiede alla View la data di restituzione effettiva.
- **Delega:** Il Controller delega l'operazione al PrestitoService

2. Chiusura e Persistenza del Prestito

- **Aggiornamento Record:** Il PrestitoService chiude il prestito, impostando la data di restituzione effettiva su Prestito.
- **Persistenza Prestito:** Il Service salva il record Prestito aggiornato nel Repository.

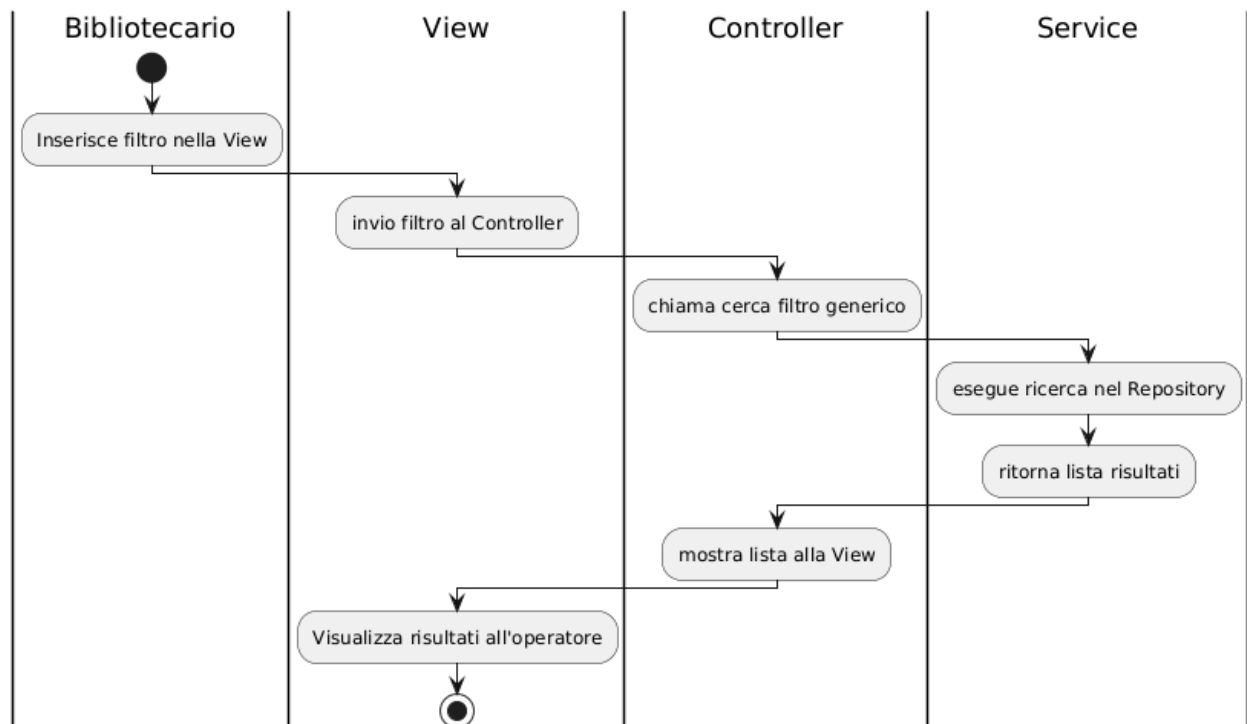
3. Aggiornamento Copie

- **Incremento Copie:** PrestitoService recupera l'oggetto Libro associato al prestito e incrementa il numero delle copie disponibili.
- **Persistenza Libro:** PrestitoService delega l'azione del salvataggio dello stato aggiornato di Libro a LibroService

4. Conclusione

1. **Notifica:** Il flusso torna al PrestitoController che tramite BaseController aggiorna la view e mostra l'esito
2. Il processo si conclude.

Ricerca View



1. Avvio e Raccolta Dati

- **Punto di partenza:** il Bibliotecario inserisce il filtro in base al quale intende effettuare la ricerca nella View, avviando il processo.
- **Delegazione:** La View riceve il filtro inserito e ne effettua l'invio al Controller.

2. Orchestrazione

- **Logica di Controllo:** Il Controller riceve il filtro passato dalla view, senza tuttavia doverlo impiegare direttamente per effettuare la ricerca.
- **Invocazione del Service:** Il Controller effettua la chiamata alla funzione di ricerca generica sul Service.

3. Logica di Business e Accesso Dati

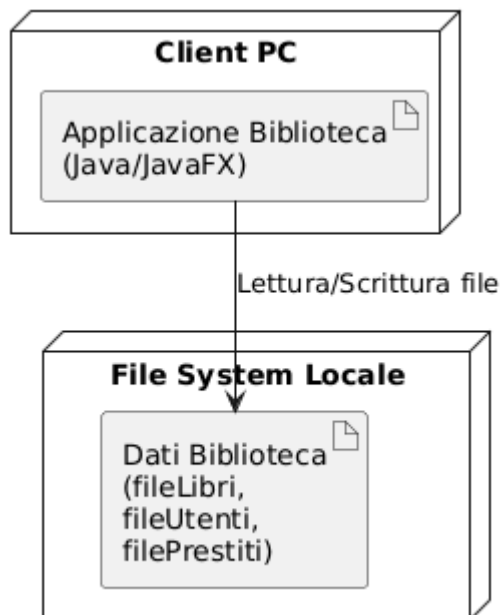
- **Implementazione della funzionalità di ricerca:** Il Service riceve la richiesta di effettuare la ricerca, occupandosene dell'effettiva implementazione. Tuttavia, l'accesso ai dati viene delegato al Repository.
- **Interazione con i Dati:** Il Repository effettua l'accesso al file, cercando i dati richiesti.
- **Restituzione dei Risultati:** Il Repository termina la ricerca ed il Service ritorna la lista dei risultati al Controller.

4. Presentazione dei Risultati

- **Aggiornamento della View:** Il Controller riceve la lista dei risultati dal Service e procede ad effettuare l'invio alla View, mediante l'azione mostra lista alla View.
- **Output Finale:** La View gestisce l'attività finale di Visualizzazione dei risultati, portando a termine il ciclo e rendendo l'output visibile al Bibliotecario.

3.3 Diagramma di distribuzione

Il Diagramma di Distribuzione è un diagramma strutturale che modella l'architettura fisica del sistema.



Il Client PC ospita l'applicazione Biblioteca, realizzata in Java. Il salvataggio dei dati avviene in 3 file diversi, memorizzati all'interno del file system locale.

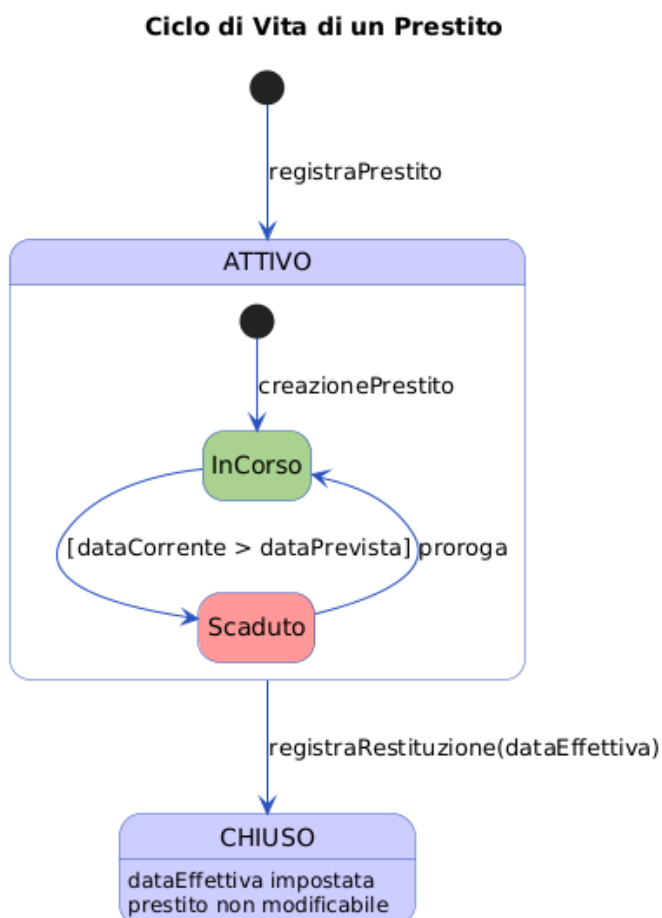
3.4 Diagramma di stati

Il Diagramma degli Stati modella il comportamento dinamico di un singolo oggetto (un'istanza di una classe) in risposta a eventi esterni e interni.

Essenzialmente, definisce tutti i possibili stati in cui un oggetto può trovarsi, come un evento provoca un cambiamento di stato e quali azioni vengono eseguite in quel momento.

Stati e Flussi del Ciclo di Vita Prestito

Il prestito passa attraverso tre stati principali, in base alla data in cui avviene la restituzione.



1. In Corso :

(Stato iniziale)

- **Condizione:** La data di restituzione effettiva (**dataEffettiva**) non è stata ancora inserita, per cui viene inizializzata al valore null. Il prestito è attivo.
- **Ingresso:** Si giunge allo stato Creato all'avvio (**Prestito(Utente, Libro, dataPrevista)**) o (**Prestito(Utente, Libro, dataPrevista, dataPrestito)**)
- **Transizioni:**
 - **A Scaduto:** Si passa allo stato **Scaduto** se il sistema verifica che la sia soddisfatta la condizione **isScaduto() == true**, che si verifica quando la data odierna è cronologicamente successiva rispetto alla **dataPrevista**.
 - **A Concluso:** Il passaggio allo stato **Concluso** avviene se la restituzione del libro avviene in tempo, attraverso l'invocazione del metodo **registraRestituzione(data)**.

2. Scaduto (In Ritardo) :

(Eventuale stato intermedio del prestito)

- **Condizione:** La data odierna è successiva alla **dataPrevista**, e la restituzione del libro non è ancora stata effettuata.
- **Transizione:** Il prestito passa dallo stato **Scaduto** allo stato **Concluso**, mediante l'invocazione del metodo **registraRestituzione(data)**. La restituzione risulta in ritardo.

3. Concluso (Restituito)

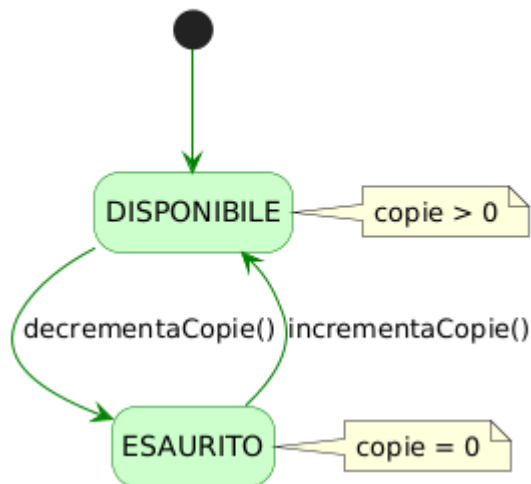
(Stato finale del prestito)

- **Condizione:** La data di restituzione effettiva (**dataEffettiva**) è stata impostata, ed il suo valore è diverso da null.
- **Ingresso:** Tale stato è raggiungibile da due stati alternativi: **InCorso**, se la restituzione avviene in tempo, o **Scaduto**, se la restituzione avviene in ritardo.
- **Uscita:** Il prestito è concluso.

Stati e Flussi di Disponibilità Libro

Il sistema modella la disponibilità di un oggetto Libro, basandosi principalmente sulla quantità di copie a disposizione e sulla validità dei dati inseriti.

Stato di Disponibilità del Libro



1. DISPONIBILE:

(Stato ordinario in cui si trova un oggetto Libro)

- **Condizione:** Sono presenti copie del libro, il che equivale al soddisfacimento della condizione **copie > 0**. In tal caso, il libro è disponibile al prestito.
- **Ingresso:** Si giunge allo stato **DISPONIBILE** all'avvio, mediante la creazione di un nuovo libro, ossia tramite **new Libro(...)**, o quando una copia viene restituita.
- **Azioni:**
 - **Prestito:** l'invocazione del metodo **decrementaCopie()** riporta allo stato **DISPONIBILE** se il numero di copie resta positivo.
 - **Restituzione:** l'invocazione del metodo **incrementaCopie()** dallo stato **ESAURITO**, riporta allo stato **DISPONIBILE**.

2. ESAURITO:

(Non sono fisicamente disponibili copie del libro da concedere in prestito)

- **Transizione:** Il libro passa dallo stato **DISPONIBILE** allo stato **ESAURITO** nel momento in cui l'invocazione del metodo **decrementaCopie()** è tale da azzerare il numero di copie disponibili.
- **Condizione:** Non sono presenti copie disponibili, il che equivale a soddisfare la condizione **copie == 0**. Non risulta possibile concedere nuovi prestiti.
- **Uscita:**
 - **Ritorno:** Il passaggio dallo stato **ESAURITO** allo stato **DISPONIBILE** avviene attraverso la chiamata del metodo **incrementaCopie()**, invocato al momento della restituzione di una copia.

3. Non Valido:

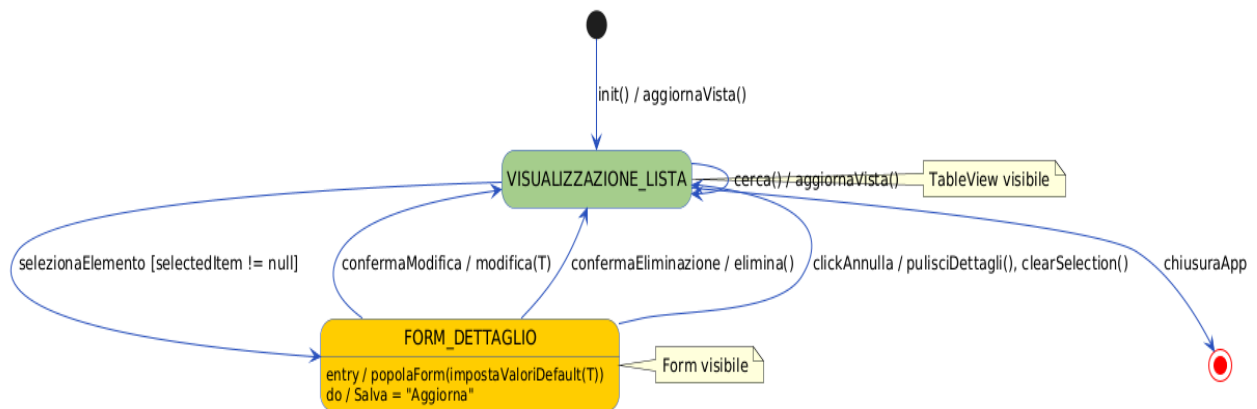
(Tale flusso si occupa di gestire i libri i cui dati risultano non validi)

- **Transizione:** Il passaggio dallo stato **Disponibile** allo stato **Non Valido** avviene se risulta soddisfatta la condizione **isValido() == false**.
- **Condizione:** Un attributo obbligatorio non è conforme(anno futuro, isbn mancante,...).
- **Uscita:** Un libro non valido viene rimosso dal ciclo operativo.

Stati e Flussi del Controller CRUD

Il sistema passa attraverso quattro stati principali, guidati dalle azioni intraprese dall'utente.

Diagramma di Stato: Flusso Logico CRUD Controller



1. Visualizzazione Lista:

(Stato iniziale dell'applicazione, nonché il principale)

- **Ingresso:** Si giunge allo stato **Visualizzazione Lista** all'avvio, mediante il metodo **init()**. Il controller adopera l'interfaccia **InterfaceView** per mostrare i dati caricati.
- **Azioni:** L'amministratore può eseguire varie operazioni, quali l'aggiornamento della vista tramite il metodo **aggiornaVista()** o **pulisciDettagli()**, la ricerca o l'eliminazione degli elementi, senza portare ad una variazione dello stato corrente.

2. Form Creazione:

(Stato di creazione di un nuovo elemento, che porta al passaggio dalla visualizzazione della lista alla visualizzazione del form di creazione)

- **Condizione:** Un elemento è selezionato nella **TableView**. I campi del form sono popolati con i dati dell'elemento.
- **Transizione:** Avviene selezionando una riga nella lista.
- **Uscita:**
 - **Successo:** L'operazione di **modifica(elemento)** o **elimina()** ha successo e riporta il sistema allo stato **VISUALIZZAZIONE LISTA**, tramite **aggiornaVista()**.

- **Annullamento:** Il pulsante "Annulla" cioè `getAnnullaButton()` invoca `pulisciDettagli()`, che **deseleziona** la riga e riporta il sistema allo stato **VISUALIZZAZIONE LISTA / FORM CREAZIONE**.

4. Coesione e Accoppiamento

La **Coesione** è una misura interna e indica quanto sono strettamente correlate e focalizzate le responsabilità di un modulo o di una classe.

L'**Accoppiamento** è una misura esterna e indica il grado di interdipendenza o di legame tra moduli, componenti o classi diverse.

4.1 Livelli di Coesione

Model

Classe/Interfaccia	Livello di Coesione	Giustificazione del livello di coesione attribuito
InterfaceID	Funzionale	L'interfaccia presenta esclusivamente un metodo, pertanto effettua un'unica operazione.
Dati	Funzionale	La classe astratta raggruppa funzionalità legate alla rappresentazione.

Libro	Funzionale	La totalità dei metodi della classe è finalizzata alla gestione dell'entità Libro.
Utente	Funzionale	Tutti i metodi della classe sono adibiti alla gestione dell'entità Utente.
Prestito	Funzionale	Tutti i metodi sono preposti alla rappresentazione dell'entità Prestito.

Repository

Classe/Interfaccia	Livello di Coesione	Giustificazione del livello di coesione attribuito
InterfaceGestoreIO<T>	Funzionale	I metodi messi a disposizione dall'interfaccia hanno il singolo compito di astrarre la persistenza: per salvare e caricare una lista di oggetti.
GestoreFile<T>	Funzionale	Tutti i metodi hanno il singolo compito di astrarre la persistenza: per salvare e caricare una lista di oggetti.
InterfaceRepository<T>	Comunicazionale	I metodi messi a disposizione dall'interfaccia operano tutti sulla medesima struttura dati.
Repository<T>	Comunicazionale	Tutti i metodi forniti dalla classe agiscono sulla medesima struttura dati.

Service

Classe/Interfaccia	Livello di Coesione	Giustificazione del livello di coesione attribuito
InterfaceService<T>	Funzionale	L'interfaccia mette a disposizione unicamente metodi che effettuano operazioni sul repository.
BaseService<T>	Funzionale	La classe fornisce unicamente metodi che effettuano operazioni sul repository.
LibroService	Comunicazionale	Tutti i metodi forniti dalla classe agiscono sulla medesima struttura dati.
UtenteService	Comunicazionale	Tutti i metodi forniti dalla classe agiscono sulla medesima struttura dati.
PrestitoService	Sequenziale	La classe mette a disposizione metodi correlati da una sequenza temporale fissa

View

Classe/Interfaccia	Livello di Coesione	Giustificazione del livello di coesione attribuito
InterfaceView<T>	Comunicazionale	Tutti i metodi messi a disposizione dall'interfaccia implementano funzionalità che lavorano sugli stessi dati

BaseView<T>	Comunicazionale	Tutti i suoi metodi implementano funzionalità che lavorano sugli stessi dati
DatiBaseView<T>	Comunicazionale	Implementa un form e una tabella. I suoi metodi implementano funzionalità che lavorano sugli stessi dati
LibroView	Comunicazionale	I metodi messi a disposizione della classe operano sugli stessi campi di input
UtenteView	Comunicazionale	I metodi messi a disposizione della classe operano sugli stessi campi di input
PrestitoView	Comunicazionale	I metodi messi a disposizione della classe operano sugli stessi campi di input

Controller

Classe/Interfaccia	Livello di Coesione	Giustificazione del livello di coesione attribuito
InterfaceController	Funzionale	I metodi messi a disposizione dall'interfaccia hanno lo scopo di inizializzare e aggiornare la visualizzazione.
BaseController<T>	Funzionale	La classe mette a disposizione metodi adibiti al coordinamento della view e del service, realizzando un singolo compito astratto.
CrudController<T>	Comunicazionale	La classe mette a disposizione metodi che lavorano sulla stessa entità.
LibroController	Comunicazionale	La totalità dei metodi messi a disposizione dalla classe che lavorano sulla stessa entità.
UtenteController	Comunicazionale	La totalità dei metodi messi a disposizione dalla classe che lavorano sulla stessa entità.
PrestitoController	Sequenziale	I metodi forniti dalla classe implementano funzionalità che devono avvenire in un ordine specifico.

Main

Main	Temporale	Il modulo raggruppa operazioni diverse che operano nello stesso arco temporale (l'avvio).
------	-----------	---

4.2 Livelli di accoppiamento

Accoppiamento per Dati

Lo scambio di informazioni tra moduli coinvolge solo informazioni strettamente necessarie al funzionamento del sistema. Rappresenta il massimo livello di accoppiamento possibile.

REPOSITORY:

- **InterfaceRepository<T> e Repository<T>:**
 - **void elimina(String id):** Il metodo richiede unicamente un parametro di input, rappresentato dall'identificativo univoco dell'elemento di cui occorre effettuare l'eliminazione.
 - **T cerca(String id):** L'identificativo univoco dell'elemento da cercare costituisce l'unico parametro necessario al metodo per effettuare la ricerca dell'elemento desiderato.

SERVICE:

- **InterfaceService<T> e BaseService<T>:**
 - **T cerca(String id):** Il metodo acquisisce in input il solo id dell'elemento di cui si deve effettuare la ricerca.
 - **List<T> cercaGenerico(String filtro):** Il parametro di input del metodo è rappresentato unicamente dalla stringa adoperata per filtrare opportunamente i risultati di ricerca.
- **LibroService:**
 - **List<Libro> cercaPerTitolo(String titolo):** Il metodo prende in ingresso unicamente il titolo del libro, ai fini della ricerca.

- **UtenteService:**

- **UtenteService: List<Utente> cercaPerCognome(String cognome):** Il metodo ha un solo parametro di input, rappresentato dal cognome dell'utente di cui effettuare la ricerca.

VIEW:

- **InterfaceView<T> e BaseView<T>:**

- **void mostraMessaggio(String messaggio):** Il metodo acquisisce in input il solo testo da visualizzare.

CONTROLLER:

- **BaseController<T>:**

- **void eseguiOperazione(Runnable operazione, String messaggioConferma):** Il metodo prende esclusivamente in ingresso l'operazione specifica da effettuare e la stringa che segnala l'esito.

Accoppiamento per timbro (Stamp coupling)

Lo scambio di informazioni tra moduli coinvolge anche informazioni che non sono strettamente necessarie. Rappresenta il secondo livello di accoppiamento più alto possibile.

MODEL:

- **Prestito:**

- **Costruttori e String getId():** I metodi mantengono operazioni e operano su Utente e Libro.

REPOSITORY:

- **InterfaceGestoreIO<T> e GestoreFile<T>:**
 - **void salvaDati(String nomeFile, List<T> dati):** Il metodo prende come parametro di ingresso unicamente il nome del file su cui effettuare il salvataggio e i dati da salvare.
 - **List<T> caricaDati(String nomeFile):** Il metodo prende come unico parametro di ingresso il nome del file su cui sono salvati i dati.
- **InterfaceRepository<T> e Repository<T>:**
 - **void inserisciOAggiorna(T elemento):** La funzionalità di aggiornamento, implementata dal metodo, necessita del passaggio di un'istanza della classe T, sebbene possa esser sufficiente aggiornare solo una parte dei valori degli attributi dell'oggetto.

SERVICE:

- **InterfaceService<T> e BaseService<T>:**
 - **void salva(T elemento):** Il metodo riceve in ingresso un'istanza della classe T. Tuttavia, il salvataggio potrebbe non essere necessario per la totalità degli attributi dell'oggetto.
 - **void modifica(T elemento):** Il metodo riceve in ingresso un oggetto di tipo elemento, provvisto di tutti gli attributi e di tutti i metodi

della classe di appartenenza. Tuttavia, la modifica può non interessare tutti gli attributi dell'oggetto passato in input.

- **PrestitoService:**

- **void registraPrestito(Utente utente, Libro libro, LocalDate dataPrevista):** Vengono passate le intere strutture Utente e Libro per ottenere i rispettivi id e manipolare le copie, nonostante non tutti i loro campi siano necessari.
- **void registraRestituzione(Prestito prestito, LocalDate dataEffettiva):** Viene passato l'intero oggetto Prestito, sebbene si debba modificare un solo campo e recuperare il Libro associato.

- **LibroService e UtenteService:**

- **elimina(T elemento):** Il metodo riceve l'intera istanza T quando solo l'id sarebbe necessario per delegare l'eliminazione al repository.

VIEW:

- **InterfaceView<T> e CrudController<T>:**

- **void mostraLista(List<T>):** Passaggio di List<T>, una struttura dati complessa.
- **T getElementoSelezionato():** Restituisce l'intera istanza T.

- **LibroView:**

- **Libro getLibroModificato() e Libro getLibroNuovo():** Il metodo visualizza l'intero libro in seguito alla procedura di modifica, includendo anche i dati sono rimasti invariati.

CONTROLLER:

- **BaseController<T> e CrudController<T>:**
 - **tutti i metodi CRUD (salva(T) e modifica(T)):** Tutti i metodi ricevono o manipolano l'intera istanza T.
- **PrestitoController:**
 - **registraPrestito() e registraRestituzione():** Le chiamate ai metodi del Service delegano il passaggio di oggetti Prestito, Utente e Libro.

5. Matrici di Tracciabilità

Libro

Componente	Moduli di test	ID dei requisiti correlati
boolean isValido()	Caso di test 11 Caso di test 14 Caso di test 15 Caso di test 16 Caso di test 12 Caso di test 13	IF-1.0.2
String getId()	Caso di test 2	DF-1.0
void decrementaCopie()	Caso di test 8 Caso di test 9	BF-3.0
void incrementaCopie()	Caso di test 10	BF-3.1
String toString()	Caso di test 17	DF-1.0, IS-1.0.3

public Libro(String titolo, List<String> autori, int anno, String isbn, int copie)	Caso di test 1	BF-1.0, IF-1.0.1, IS-1.0.3
public List<String> getAutori()	Caso di test 3 Caso di test 18	DF-1.0, UI-1.0, IF-1.2.2
public void setTitolo(String nuovoTitolo)	Caso di test 4	IF-1.0.1
public void setAutori(List<String> nuovoAutori)	Caso di test 19	IF-1.0.1
public void setAnno(int nuovoAnno)	Caso di test 5	IF-1.0.1
public void setCopieDisponibili(int nuovoCopie)	Caso di test 6 Caso di test 7	IF-1.0.1, FC-3.0.1

Elenco casi di test per la classe Libro (metodi di LibroTest)

- Caso di test 1: void testCostruttoreEGetter()
- Caso di test 2: void testGetId()
- Caso di test 3: void testGetAutoriString()
- Caso di test 4: void testSetTitolo()
- Caso di test 5: void testSetAnno()
- Caso di test 6: void testSetCopieDisponibili()
- Caso di test 7: void testSetCopieDisponibili_Nessuna()
- Caso di test 8: void testDecrementaCopie()
- Caso di test 9: void testDecrementaCopie_LimiteZero()
- Caso di test 10: void testIncrementaCopie()
- Caso di test 11: void testIsValido()
- Caso di test 12: void testIsValido_TitoloMancante()
- Caso di test 13: void testIsValido_AutoriMancanti()
- Caso di test 14: void testIsValido_AnnoNonCorretto()
- Caso di test 15: void testIsValido_CopieNegative()
- Caso di test 16: void testIsValido_IsbnNullo()
- Caso di test 17: void testToString_FormatoCorretto()
- Caso di test 18: void testGetAutori()
- Caso di test 19: void testSetAutori()

Prestito

Componente	Moduli di test	ID dei requisiti correlati
------------	----------------	----------------------------

void registraRestituzione(LocalDate data)	Caso di test 24 Caso di test 25 Caso di test 34	BF-3.0
boolean isScaduto()	Caso di test 26 Caso di test 27 Caso di test 28 Caso di test 29	FC-3.1.1
String getId()	Caso di test 30 Caso di test 35	DF-3.0, IS-3.0.3
String toString()	Caso di test 32 Caso di test 33	IS-3.0.3, DF-3.0
public Prestito(Utente utente, Libro libro, LocalDate dataPrevista)	Caso di test 36	BF-3.0
public Prestito(Utente utente, Libro libro, LocalDate dataPrevista, LocalDate dataPrestito)	Caso di test 20 Caso di test 21 Caso di test 22 Caso di test 23 Caso di test 36	BF-3.0
void getNomeUtente()	Caso di test 37 Caso di test 31	DF-2.0, FC-2.2.2, UI-2.0
void getTitoloLibro()	Caso di test 38 Caso di test 31	DF-1.0, IF-1.2.1, FC-1.2.1, UI-1.0
void getDataPrestito()	Caso di test 39	DF-2.0, DF-2.0, FC-3.1.1, IF-3.1.2
void getDataPrevista()	Caso di test 40	DF-2.0, DF-2.0, FC-3.1.1
void getDataEffettiva()	Caso di test 41	IF-3.1.2
void getUtente()	Caso di test 42	DF-3.0
void getLibro()	Caso di test 43	DF-3.0

Elenco casi di test per la classe Prestito (metodi di PrestitoTest)

- Caso di test 20: void testCostruttore()
- Caso di test 21: void testCostruttore_UtenteNullo()
- Caso di test 22: void testCostruttore_LibroNullo()
- Caso di test 23: void testCostruttore_DataPrestitoNulla()
- Caso di test 24: void testRegistraRestituzione()
- Caso di test 25: void testRegistraRestituzione_DataNulla()

- Caso di test 26: void testIsScaduto()
- Caso di test 27: void testIsScaduto_GiornoDiScadenza()
- Caso di test 28: void testIsScaduto_NonScaduto()
- Caso di test 29: void testIsScaduto_DopoRestituzione()
- Caso di test 30: void testGetId()
- Caso di test 31: void testGetter()
- Caso di test 32: void testToString_NotNull()
- Caso di test 33: void testToString_DopoRestituzione()
- Caso di test 34: void testRestituzionePrestito_InAnticipo()
- Caso di test 35: void testGetId_Unico()
- Caso di test 36: void testCostruttore_DefaultDataPrestito()
- Caso di test 37: void testGetNomeUtente()
- Caso di test 38: void testGetTitoloLibro()
- Caso di test 39: void testGetDataPrestito()
- Caso di test 40: void testGetDataPrevista()
- Caso di test 41: void testGetDataEffettiva_Activo()
- Caso di test 42: void testGetUtente()
- Caso di test 43: void testGetLibro()

Utente

Componente	Moduli di test	ID dei requisiti correlati
boolean isValido()	Caso di test 52	IF-2.0.2
String getId()	Caso di test 45	DF-2.0
String toString()	Caso di test 55	DF-2.0, IS-2.0.3
public Utente(String nome, String cognome, String matricola, String email)	Caso di test 44 Caso di test 50 Caso di test 54 Caso di test 53	BF-2.0, IS-2.0.3, IF-2.0.1
public String getNome()	Caso di test 57	DF-2.0, FC-2.2.2
public String getCognome()	Caso di test 58	DF-2.0, FC-2.2.1
public String getMatricola()	Caso di test 59	DF-2.0, IF-2.0.1
public String getEmail()	Caso di test 60	DF-2.0, IF-2.0.1

public void setNome(String nuovoNome)	Caso di test 46	IF-2.0.1
public void setCognome(String nuovoCognome)	Caso di test 47	IF-2.0.1
public void setMatricola(String nuovaMatricola)	Caso di test 49 Caso di test 56	IF-2.0.1
public void setEmail(String nuovaEmail)	Caso di test 48 Caso di test 51	IF-2.0.1

Elenco casi di test per la classe Utente (metodi di UtenteTest)

- Caso di test 44: testCostruttoreEGetter()
- Caso di test 45: testGetID()
- Caso di test 46: testSetNome()
- Caso di test 47: testSetCognome()
- Caso di test 48: testSetEmail()
- Caso di test 49: testSetMatricola()
- Caso di test 50: testCostruttore_EmailNonConformeLanciaEccezione()
- Caso di test 51: testSetEmail_NonConformeLanciaEccezione()
- Caso di test 52: testIsValido()
- Caso di test 53: testCostruttore_UtenteSenzaNomeLanciaEccezione()
- Caso di test 54: testCostruttore_UtenteSenzaMatricolaLanciaEccezione()
- Caso di test 55: testToString()
- Caso di test 56: testSetMatricola_MatricolaNullaOVuota()
- Caso di test 57: testGetNome()
- Caso di test 58: testGetCognome()
- Caso di test 59: testGetMatricola()
- Caso di test 60: testGetEmail()

GestoreFile

Componente	Moduli di test	ID dei requisiti correlati
void salvaDati(String nomeFile, List<T> dati)	Caso di test 61 Caso di test 62 Caso di test 63	IS-1.0.3, IS-2.0.3, IS-3.0.3,
List<T> caricaDati(String nomeFile)	Caso di test 64 Caso di test 65 Caso di test 66	IS-1.0.3, IS-2.0.3, IS-3.0.3

Elenco casi di test per la classe GestoreFile (metodi di GestoreFileTest)

- Caso di test 61: testSalvaECaricaDati()
- Caso di test 62: testSalvaListaVuota()
- Caso di test 63: testSalvaListaNull()
- Caso di test 64: testSovrascritturaFile()
- Caso di test 65: testCaricaDati()
- Caso di test 66: testCaricaFileInesistente()

Repository

Componente	Moduli di test	ID dei requisiti correlati
void caricaTutti(List<T> lista)	Caso di test 83	IS-1.0.3, IS-2.0.3, IS-3.0.3
void salvaSuFile()	Caso di test 82	IS-1.0.3, IS-2.0.3, IS-3.0.3
void inserisciOAggiorna(T elemento)	Caso di test 69 Caso di test 70 Caso di test 71 Caso di test 72	IF-1.0.1, IF-2.0.1, IF-3.0.2, IF-1.1.1, IF-2.1.1
public void elimina(String id)	Caso di test 73 Caso di test 74 Caso di test 75	IF-1.1.2, IF-2.1.2
public T cerca(String id)	Caso di test 76 Caso di test 77 Caso di test 78	IF-1.2.1, IF-1.2.2, IF-1.2.3, IF-2.2.1, IF-2.2.2
public List<T> getAll()	Caso di test 79 Caso di test 80 Caso di test 81	UI-1.0, UI-2.0, UI-3.0
public Repository(String nomeFile, InterfaceGestoreIO<T> gestoreIO)	Caso di test 67 Caso di test 68	IS-1.0.3, IS-2.0.3, IS-3.0.3

Elenco casi di test per la classe Repository (metodi di RepositoryTest)

- Caso di test 67: testCostruttore_CaricamentoDatiIniziali()
- Caso di test 68: testCostruttore_InizializzazioneVuota()
- Caso di test 69: testInserisci_NuovoElemento()
- Caso di test 70: testInserisci_AggiornamentoElemento()

- Caso di test 71: testInserisci_ElementoNull()
- Caso di test 72: testInserisci_ElementoConIdNull()
- Caso di test 73: testElimina_ElementoEsistente()
- Caso di test 74: testElimina_ElementoInesistente()
- Caso di test 75: testElimina_IdNull()
- Caso di test 76: testCerca_ElementoPresente()
- Caso di test 77: testCerca_ElementoAssente()
- Caso di test 78: testCerca_IdNull()
- Caso di test 79: testGetAll_ListaVuota()
- Caso di test 80: testGetAll_ListaRiempita()
- Caso di test 81: testGetAll_Incapsulamento()
- Caso di test 82: testSalvaSuFile()
- Caso di test 83: testCaricaTutti()

LibroService

Componente	Moduli di test	ID dei requisiti correlati
public List<Libro> cercaPerTitolo(String titolo)	Caso di test 88 Caso di test 89 Caso di test 90 Caso di test 91 Caso di test 92	IF-1.2, BF-1.2
public List<Libro> cercaPerAutore(String autore)	Caso di test 96 Caso di test 93 Caso di test 94 Caso di test 95 Caso di test 97 Caso di test 98	IF-1.2.2, BF-1.2
public void elimina(Libro elemento)	Caso di test 99 Caso di test 100 Caso di test 103	IF-1.1.2, BF-1.1
public List<Libro> cercaGenerico(String filtro)	Caso di test 85 Caso di test 86 Caso di test 87	IF-1.2.1, IF-1.2.2, IF-1.2.3
public LibroService(InterfaceRepository<Libro> repository)	Caso di test 84	BF-1.0
T cerca(String id)	Caso di test 101 Caso di test 102	IF-1.2.3, IF-2.2.2
void salva(T elemento)	Caso di test 102	IS-1.0.3, IS-2.0.3, IS-3.0.3

Elenco casi di test per la classe LibroService (metodi di LibroServiceTest)

- Caso di test 84: testCostruttore()
- Caso di test 85: testCercaGenerico_MatchTitoloAutore()
- Caso di test 86: testCercaGenerico_MatchISBN()
- Caso di test 87: testCercaGenerico_FiltroVuotoONullo_RestituisceTutti()
- Caso di test 88: testCercaPerTitolo_MatchSingolo()
- Caso di test 89: testCercaPerTitolo_MatchMultiplo()
- Caso di test 90: testCercaPerTitolo_NessunMatch()
- Caso di test 91: testCercaPerTitolo_FiltroCaseInsensitive()
- Caso di test 92: testCercaPerTitolo_FiltroNullo_ComportamentoAtteso()
- Caso di test 93: testCercaPerAutore_MatchSingoloAutoreCompleto()
- Caso di test 94: testCercaPerAutore_MatchMultiplo_NomeParziale()
- Caso di test 95: testCercaPerAutore_MatchMultiplo_StessoAutore()
- Caso di test 96: testCercaPerAutore_NessunMatch()
- Caso di test 97: testCercaPerAutore_FiltroCaseInsensitive()
- Caso di test 98: testCercaPerAutore_FiltroVuotoONullo_ComportamentoAtteso()
- Caso di test 99: testElimina_PrestitiAttiviImpedisconoEliminazione()
- Caso di test 100: testElimina_NessunPrestitoAttivo_Successo()
- Caso di test 101: testCerca_IdNulloOVuoto()
- Caso di test 102: testSalva_ElementoNullo()
- Caso di test 103: testElimina_ElementoNullo()

PrestitoService

Componente	Moduli di test	ID dei requisiti correlati
public void registraPrestito(Utente utente, Libro libro, LocalDate dataPrevista)	Caso di test 108 Caso di test 109 Caso di test 110 Caso di test 111 Caso di test 112 Caso di test 120	BF-3.0
public void registraRestituzione(Prestito prestito, LocalDate dataEffettiva)	Caso di test 113 Caso di test 114 Caso di test 115	BF-3.1

public List<Prestito> listaPrestitiAttivi()	Caso di test 116 Caso di test 117 Caso di test 118 Caso di test 119	UI-3.0, BF-3.0, BF-3.1
public List<Prestito> cercaGenerico(String filtro)	Caso di test 106 Caso di test 105 Caso di test 107	FC-3.1.1, BF-3.1
public PrestitoService(InterfaceRepository <Prestito> repository, LibroService libroService)	Caso di test 104	BF-3.0

Elenco casi di test per la classe PrestitoService (metodi di PrestitoServiceTest)

- Caso di test 104: testCostruttore()
- Caso di test 105: testCercaGenerico_MatchNomeUtente()
- Caso di test 106: testCercaGenerico_MatchTitoloLibro()
- Caso di test 107: testCercaGenerico_FiltroNullo_RestituisceTutti()
- Caso di test 108: testRegistraPrestito()
- Caso di test 109: testRegistraPrestito_LibroConPocheCopie()
- Caso di test 110: testRegistraPrestito_UtenteNullo()
- Caso di test 111: testRegistraPrestito_LibroNullo()
- Caso di test 112: testRegistraPrestito_DataPrevistaPassata()
- Caso di test 113: testRegistraRestituzione()
- Caso di test 114: testRegistraRestituzione_PrestitoGiaChiuso()
- Caso di test 115: testRegistraRestituzione_PrestitoNullo()
- Caso di test 116: testListaPrestitiAttivi_UnoAttivo()
- Caso di test 117: testListaPrestitiAttivi_NessunAttivo()
- Caso di test 118: testListaPrestitiAttivi_Misti()
- Caso di test 119: testListaPrestitiAttivi_RepositoryVuoto()
- Caso di test 120: testRegistraPrestito_LimiteSuperato()

UtenteService

Componente	Moduli di test	ID dei requisiti correlati
public void elimina(Utente elemento)	Caso di test 135 Caso di test 136	IF-2.1.2
public List<Utente> cercaPerCognome(String cognome)	Caso di test 125 Caso di test 126 Caso di test 127 Caso di test 128 Caso di test 129 Caso di test 130	IF-2.2.1

	Caso di test 131 Caso di test 132 Caso di test 133 Caso di test 134	
public List<Utente> cercaGenerico(String filtro)	Caso di test 122 Caso di test 123 Caso di test 124	IF-2.2.1, IF-2.2.2
public UtenteService(InterfaceRepository< Utente> repository, PrestitoService prestitoService)	Caso di test 121	BF-2.0

Elenco casi di test per la classe UtenteService (metodi di UtenteServiceTest)

- Caso di test 121: testCostruttore()
- Caso di test 122: testCercaGenerico_MatchCognome()
- Caso di test 123: testCercaGenerico_MatchMatricola()
- Caso di test 124: testCercaGenerico_FiltroVuotoONullo_RestituisceTutti()
- Caso di test 125: testCercaPerCognome_MatchMultiplo_Nessuno()
- Caso di test 126: testCercaPerCognome_MatchSingolo()
- Caso di test 127: testCercaPerCognome_NessunMatch()
- Caso di test 128: testCercaPerCognome_FiltroNullo()
- Caso di test 129: testCercaPerCognome_CaseSensitivityEsatta()
- Caso di test 130: testCercaPerCognome_CaseSensitivityMinuscola()
- Caso di test 131: testCercaPerCognome_FiltroVuoto()
- Caso di test 132: testCercaPerCognome_FiltroSpazi()
- Caso di test 133: testCercaPerCognome_AltroSingolo()
- Caso di test 134: testCercaPerCognome_AltroSingoloMinuscolo()
- Caso di test 135: testElimina_PrestitiAttiviImpedisconoEliminazione()
- Caso di test 136: testElimina_NessunPrestitoAttivo()

6. Mock-up dell'Interfaccia Utente

L'interfaccia grafica è strutturata in un'unica finestra divisa in tre schede, ognuna che si occupa della gestione di Libro, Utente e Prestiti.

Le interazioni seguono il modello standard CRUD (Create,Read,Update,Delete) con alcune modifiche aggiuntive per la sezione Prestiti.

Sistema Gestione Biblioteca

Gestione Libri

Gestione Utenti

Gestione Prestiti

Cerca:

Annulla Cerca

Elimina

Titolo	Autori	Anno	ISBN	Copie disponibili
Il Codice Da Vinci	Dan Brown	2003	978-8804526211	2
Guerra e Pace	Lev Tolstoj	1869	978-8806208882	0
Fisica per l'università	Halliday, Resnick	2018	978-8808182740	4
Un'estate italiana	Marco Rossi	2021	978-8832903247	2
Clean Code	Robert C. Martin	2008	978-0132350884	4
Design Patterns	Gamma, Helm, Joh...	1994	978-0201633610	2
The Lord of the Rings	J.R.R. Tolkien	1954	978-0618574940	4
Java in a Nutshell	David Flanagan	2020	978-1492076260	2
L'ombra del vento	Carlos Ruiz Zafón	2001	978-8804510746	2
Artificial Intelligence	Stuart Russell, Peter...	2023	978-0134610993	4

Dettagli Libro

Titolo:

Anno:

ISBN:

Copie disponibili:

Pronto.

Salva/Aggiorna

Annulla

Sistema Gestione Biblioteca

Gestione Libri

Gestione Utenti

Gestione Prestiti

Cerca:

Annulla Cerca

Elimina

Nome	Cognome	Matricola	Email
Anna	Bianchi	A0002	anna.bianchi@example.com
Giulia	Bruni	G0006	giulia.bruni@example.com
Andrea	Ferrari	A0009	andrea.ferrari@example.com
Chiara	Gallo	C0010	chiara.gallo@example.com
Paolo	Gialli	P0005	paolo.gialli@example.com
Davide	Moretti	D0007	davide.moretti@example.c...
Sara	Neri	S0004	sara.neri@example.com
Elena	Rizzo	E0008	elena.rizzo@example.com
Mario	Rossi	M0001	mario.rossi@example.com
Luca	Verdi	L0003	luca.verdi@example.com

Dettagli Utente

Nome:

Cognome:

Matricola:

Email:

Pronto.

Salva/Aggiorna

Annulla

Sistema Gestione Biblioteca

Gestione Libri

Gestione Utenti

Gestione Prestiti

Cerca:

Annulla Cerca

Elimina

Libro	Utente	Data prestito	Data restituzione	Stato
Guerra e Pace	Bianchi	13/12/2025	10/12/2025	SCADUTO
Java in a Nutshell	Rizzo	13/12/2025	12/12/2025	SCADUTO
The Lord of the Ri...	Moretti	13/12/2025	18/12/2025	In Prestito
Il Codice Da Vinci	Rossi	13/12/2025	20/12/2025	In Prestito
Design Patterns	Bruni	13/12/2025	23/12/2025	In Prestito
Fisica per l'univers...	Verdi	13/12/2025	27/12/2025	In Prestito
The Lord of the Ri...	Gallo	13/12/2025	02/01/2026	In Prestito

Dettagli Prestito

Libro:

Utente:

Data prestito:

14/12/2025

Data restituzione:

Stato:

In Prestito

Pronto per registrare un nuovo prestito.

Registra Prestito

Restituisci Libro

Annulla

Interazioni Comuni a tutte le schede:

1. Visualizzazione e Selezione Dati

- **Visualizzazione:** L'utente visualizza la tabella contenente tutti gli elementi dell'elenco
- **Selezione:** L'utente seleziona una riga, vengono subito mostrati gli elementi dentro il pannello dettagli, e può modificare o eliminare gli elementi della riga
- **Messaggio di errore/conferma:** piccola sezione che mostra a video i messaggi di conferma/errore.

2. Ricerca e Filtro

- **Ricerca (Dinamica):** L'utente seleziona il campo cerca e ad ogni carattere inserito vengono mostrati nella tabella solo gli elementi corrispondenti
- **Annulla Ricerca:** L'utente cliccando questa sezione svuota il textfield dai caratteri precedentemente inseriti

3. Azioni sul Dettaglio (CRUD)

- **Aggiungi Nuovo:** L'utente ,dopo avere inserito tutti i campi dentro il pannello dei dettagli, salva un nuovo elemento cliccando su salva nuovo

- **Modifica:** L'utente, dopo avere cliccato un elemento di una riga, modifica i campi selezionati e clicca aggiorna
- **Elimina:** L'utente, dopo avere cliccato un elemento di una riga, clicca su elimina
- **Annulla Modifica/Inserimento:** L'utente cliccando questa sezione svuota i textfield dai caratteri precedentemente inseriti in tutto il pannello dettagli

Interazioni specifiche per Gestione prestiti

1. Visualizzazione Iniziale

- **Ordinamento e Evidenziazione:** I prestiti vengono ordinati in base all'importanza e vengono evidenziati di rosso quelli scaduti e di verde quelli restituiti

2. Registrazione Nuovo Prestito

- **Input Obbligatorio:** L'utente seleziona un Utente e un Libro già salvati dalle rispettive ComboBox
- **Data Prevista:** L'utente seleziona la data di restituzione prevista tramite il datepicker
- **Azione:** L'utente clicca su registra prestito

3. Registrazione Restituzione (Chiusura Prestito)

- **Selezione:** L'utente seleziona un prestito attivo dalla tabella
- **Modifica Form:** Vengono disabilitati i campi Libro e utente, ma viene abilitato il pulsante restituisci libro
- **Input Data Effettiva:** L'utente inserisce la data effettiva di restituzione tramite datepicker
- **Azione:** L'utente clicca su restituisci libro